

Depuis le krach boursier de 1929 et la crise bancaire qui s'en est suivie, le besoin de renforcer la régulation du secteur financier n'a cessé de croître. Cette volonté commune des institutions internationales s'est concrétisée par la création du Comité de Bâle en 1988, chargé de définir des standards destinés à garantir la stabilité du système bancaire mondial. Ainsi, trois grands accords se sont succédé :

- Bâle I (1988) : premières règles relatives aux fonds propres;
- Bâle II (2004) : intégration du risque de crédit et du contrôle interne;
- Bâle III (2010) : réponse à la crise financière de 2008, renforçant la transparence et les exigences de capital. Parallèlement, l'entrée en vigueur de la norme IFRS 9 en 2018 a marqué un tournant décisif dans la gestion du risque de crédit. Elle introduit la notion de pertes attendues dès la reconnaissance initiale du crédit, incitant ainsi les établissements financiers à anticiper davantage le risque de défaut. Dans ce contexte, la transparence des algorithmes de scoring, la fiabilité des prédictions et la conformité réglementaire sont devenues des enjeux centraux.

Les institutions doivent désormais concilier performance prédictive, explicabilité et équité dans leurs décisions de crédit. Dès lors, une question fondamentale guide ce projet : Comment concevoir un modèle de scoring de crédit robuste, fiable et conforme aux exigences réglementaires actuelles? Ce projet vise à développer un système de scoring de crédit à la fois robuste, explicable et aligné sur les régulations (Bâle II/III, IFRS 9, GDPR).

L'objectif principal est de modéliser la probabilité de défaut (PD) des emprunteurs tout en garantissant une calibration adéquate et une équité algorithmique. Le rapport s'articule autour de cinq grandes parties, allant du cadrage métier à l'analyse de l'équité, en passant par la modélisation, la calibration et l'analyse des biais.

```
In [1]: import pandas as pd
import numpy as np

df = pd.read_csv('loans_full_schema.csv')
df.head(10)
```

Out[1]:

	emp_title	emp_length	state	homeownership	annual_income	verified_income	deb
--	-----------	------------	-------	---------------	---------------	-----------------	-----

0	global config engineer	3.0	NJ	MORTGAGE	90000.0	Verified	
---	------------------------------	-----	----	----------	---------	----------	--

1	warehouse office clerk	10.0	HI	RENT	40000.0	Not Verified	
---	------------------------------	------	----	------	---------	--------------	--

2	assembly	3.0	WI	RENT	40000.0	Source Verified	
---	----------	-----	----	------	---------	-----------------	--

3	customer service	1.0	PA	RENT	30000.0	Not Verified	
---	---------------------	-----	----	------	---------	--------------	--

4	security supervisor	10.0	CA	RENT	35000.0	Verified	
---	------------------------	------	----	------	---------	----------	--

5	NaN	NaN	KY	OWN	34000.0	Not Verified	
---	-----	-----	----	-----	---------	--------------	--

6	hr	10.0	MI	MORTGAGE	35000.0	Source Verified	
---	----	------	----	----------	---------	-----------------	--

7	police	10.0	AZ	MORTGAGE	110000.0	Source Verified	
---	--------	------	----	----------	----------	-----------------	--

8	parts	10.0	NV	MORTGAGE	65000.0	Source Verified	
---	-------	------	----	----------	---------	-----------------	--

9	4th person	3.0	IL	RENT	30000.0	Not Verified	
---	------------	-----	----	------	---------	--------------	--

10 rows × 55 columns



In [2]: `df['issue_month'].head(266)`

Out[2]:

0	Mar-2018
1	Feb-2018
2	Feb-2018
3	Jan-2018
4	Mar-2018
	...
261	Mar-2018
262	Mar-2018
263	Jan-2018
264	Jan-2018
265	Mar-2018

Name: issue_month, Length: 266, dtype: object

In [3]: `df.describe()`

Out[3]:

	emp_length	annual_income	debt_to_income	annual_income_joint	debt_to_incon
count	9183.000000	1.000000e+04	9976.000000	1.495000e+03	1495
mean	5.930306	7.922215e+04	19.308192	1.279146e+05	19
std	3.703734	6.473429e+04	15.004851	7.016838e+04	8
min	0.000000	0.000000e+00	0.000000	1.920000e+04	0
25%	2.000000	4.500000e+04	11.057500	8.683350e+04	14
50%	6.000000	6.500000e+04	17.570000	1.130000e+05	19
75%	10.000000	9.500000e+04	25.002500	1.515455e+05	25
max	10.000000	2.300000e+06	469.090000	1.100000e+06	39

8 rows × 42 columns



In []:

observation des variables

Une observation de nos variables montre la présence de plusieurs valeurs manquantes, notamment pour les variables **emp_title**, **emp_length**, **debt_to_income**, **num_accounts_120d_past_due**, mais très fréquentes sur **annual_income_joint**, **verification_income_joint**, **debt_to_income_joint**, **months_since_last_delinq**, **months_since_90d_late**, **months_since_last_credit_inquiry**.

Pour **annual_income_joint**, **verification_income_joint**, **debt_to_income_joint**, on a environ 8500 valeurs manquantes pour une base de données de 10 000 prêts octroyés. Cela pourrait signifier qu'on n'a pas beaucoup de demandes conjointes. Donc, pour traiter ces valeurs manquantes, nous nous proposons de les remplacer par zéro.

emp_title est une variable qui ne sera pas très utile dans notre analyse, car avant tout, nous ne cherchons pas les profils de métiers qui respectent leurs engagements ou non. Donc, cette variable pourrait être supprimée.

emp_length, **debt_to_income**, **num_accounts_120d_past_due** : ici, les valeurs manquantes pourront être remplacées par la moyenne des différentes variables.

months_since_last_credit_inquiry : Nombre de mois depuis la dernière demande de renseignements sur le crédit de ce demandeur : nous avons environ 5500 valeurs manquantes, ce qui peut être justifié peut-être par l'oubli des clients. Donc, ici, en remplaçant par la moyenne, nous commettons moins d'erreurs, car s'ils ont eu un prêt, cela voudrait dire qu'ils ont une historique de prêts bien gérée. Nous pouvons avoir le même raisonnement pour **months_since_90d_late** : Nombre de mois depuis la dernière fois que le demandeur a eu un retard de paiement de 90 jours (ou cela peut aussi être remplacé par 0) pareil pour **months_since_last_credit_inquiry**.

```
In [4]: nombre_de_valeurs_manquante = df.isnull().sum()
nombre_de_valeurs_manquante
```

```
Out[4]: emp_title      833
emp_length    817
state         0
homeownership 0
annual_income 0
verified_income 0
debt_to_income 24
annual_income_joint 8505
verification_income_joint 8545
debt_to_income_joint 8505
delinq_2y      0
months_since_last_delinq 5658
earliest_credit_line 0
inquiries_last_12m 0
total_credit_lines 0
open_credit_lines 0
total_credit_limit 0
total_credit_utilized 0
num_collections_last_12m 0
num_historical_failed_to_pay 0
months_since_90d_late 7715
current_accounts_delinq 0
total_collection_amount_ever 0
current_installment_accounts 0
accounts_opened_24m 0
months_since_last_credit_inquiry 1271
num_satisfactory_accounts 0
num_accounts_120d_past_due 318
num_accounts_30d_past_due 0
num_active_debit_accounts 0
total_debit_limit 0
num_total_cc_accounts 0
num_open_cc_accounts 0
num_cc_carrying_balance 0
num_mort_accounts 0
account_never_delinq_percent 0
tax_liens      0
public_record_bankrupt 0
loan_purpose      0
application_type 0
loan_amount     0
term            0
interest_rate   0
installment     0
grade          0
sub_grade       0
issue_month     0
loan_status     0
initial_listing_status 0
disbursement_method 0
balance        0
paid_total      0
paid_principal  0
paid_interest   0
paid_late_fees  0
dtype: int64
```

```
In [5]: colonnes_par_zero = ['annual_income_joint', 'verification_income_joint', 'debt_to_income_joint']
df[colonnes_par_zero] = df[colonnes_par_zero].fillna(0)

print("Valeurs manquantes après le remplacement par 0 :")
print(df[colonnes_par_zero].isnull().sum())

# Calculer la moyenne de chaque colonne et stocker les valeurs
mean_emp = df['emp_length'].mean()
mean_debt = df['debt_to_income'].mean()
mean_num = df['num_accounts_120d_past_due'].mean()
mean_months = df['months_since_last_credit_inquiry'].mean()
#mean_months_since = df['months_since_90d_late'].mean()
mean_months_credit = df['months_since_last_credit_inquiry'].mean()
mean_last_delinq = df['months_since_last_delinq'].mean()
# Remplacer les valeurs manquantes avec les moyennes respectives
df['emp_length'] = df['emp_length'].fillna(mean_emp)
df['debt_to_income'] = df['debt_to_income'].fillna(mean_debt)
df['num_accounts_120d_past_due'] = df['num_accounts_120d_past_due'].fillna(mean_num)
df['months_since_last_credit_inquiry'] = df['months_since_last_credit_inquiry'].fillna(mean_months)
#df['months_since_90d_late'] = df['months_since_90d_late'].fillna(mean_months_since)
df['months_since_last_credit_inquiry'] = df['months_since_last_credit_inquiry'].fillna(mean_months_credit)
df['months_since_last_delinq'] = df['months_since_last_delinq'].fillna(mean_last_delinq)
```

Valeurs manquantes après le remplacement par 0 :

```
annual_income_joint      0
verification_income_joint 0
debt_to_income_joint      0
months_since_90d_late    0
dtype: int64
```

```
In [6]: nombre_de_valeurs_manquante = df.isnull().sum()
print(nombre_de_valeurs_manquante)
```

emp_title	833
emp_length	0
state	0
homeownership	0
annual_income	0
verified_income	0
debt_to_income	0
annual_income_joint	0
verification_income_joint	0
debt_to_income_joint	0
delinq_2y	0
months_since_last_delinq	0
earliest_credit_line	0
inquiries_last_12m	0
total_credit_lines	0
open_credit_lines	0
total_credit_limit	0
total_credit_utilized	0
num_collections_last_12m	0
num_historical_failed_to_pay	0
months_since_90d_late	0
current_accounts_delinq	0
total_collection_amount_ever	0
current_installment_accounts	0
accounts_opened_24m	0
months_since_last_credit_inquiry	0
num_satisfactory_accounts	0
num_accounts_120d_past_due	0
num_accounts_30d_past_due	0
num_active_debit_accounts	0
total_debit_limit	0
num_total_cc_accounts	0
num_open_cc_accounts	0
num_cc_carrying_balance	0
num_mort_accounts	0
account_never_delinq_percent	0
tax_liens	0
public_record_bankrupt	0
loan_purpose	0
application_type	0
loan_amount	0
term	0
interest_rate	0
installment	0
grade	0
sub_grade	0
issue_month	0
loan_status	0
initial_listing_status	0
disbursement_method	0
balance	0
paid_total	0
paid_principal	0
paid_interest	0
paid_late_fees	0
dtype:	int64

```
for col in df.select_dtypes(include='number').columns: q1, q3 = df[col].quantile([0.25, 0.75]) iqr = q3 - q1 born_inf, born_sup
= q1 - 1.5*iqr, q3 + 1.5*iqr df[col] = np.where(df[col] > born_sup, born_sup, np.where(df[col] < born_inf, born_inf, df[col]))
```

enrichissement de la base de donnees

Afin de rendre notre analyse beaucoup plus précise, nous introduisons des **ratios financiers** avec les variables disponibles, ce qui va nous permettre de prendre en compte les risques de surendettement et les corrélations à un risque de défaut important (ou grand).

Des variables comportementales comme **credit_mix_score** et **delinquency_rate** vont nous permettre de tenir compte de l'expérience de l'emprunteur en matière de crédit, et aussi la capacité du client à respecter ses engagements, ce qui tient compte également de la stabilité financière du client. Par exemple, un score de diversité de crédit faible peut parfois être un signe d'accès limité au crédit.

Dans le même ordre d'idée, notre analyse à travers **account_stability** tiendra compte de la proportion des comptes de l'emprunteur qui n'ont jamais connu de retard. Ce qui permettrait de prendre en compte de manière beaucoup plus précise la stabilité financière du client et surtout son expérience en matière de prêt.

```
In [7]: # Ratios financiers avec Les variables disponibles
df['debt_service_ratio'] = df['installment'] / (df['annual_income'] / 12)
df['credit_utilization_rate'] = df['total_credit_utilized'] / df['total_credit_l']
df['payment_to_income'] = df['installment'] / df['annual_income']

# Variables comportementales du dataset
df['credit_mix_score'] = (df['num_mort_accounts'] + df['num_open_cc_accounts'] +
                        df['current_installment_accounts']) / df['total_credit']
df['delinquency_rate'] = df['delinq_2y'] / 2

# Taux annuel de retard
df['account_stability'] = df['account_never_delinq_percent'] / 100
```

```
In [8]: df
```

Out[8]:

	emp_title	emp_length	state	homeownership	annual_income	verified_income
0	global config engineer	3.0	NJ	MORTGAGE	90000.0	Verified
1	warehouse office clerk	10.0	HI	RENT	40000.0	Not Verified
2	assembly	3.0	WI	RENT	40000.0	Source Verified
3	customer service	1.0	PA	RENT	30000.0	Not Verified
4	security supervisor	10.0	CA	RENT	35000.0	Verified
...	...	...	...	...	...	...
9995	owner	10.0	TX	RENT	108000.0	Source Verified
9996	director	8.0	PA	MORTGAGE	121000.0	Verified
9997	toolmaker	10.0	CT	MORTGAGE	67000.0	Verified
9998	manager	1.0	WI	MORTGAGE	80000.0	Source Verified
9999	operations analyst	3.0	CT	RENT	66000.0	Not Verified

10000 rows × 61 columns



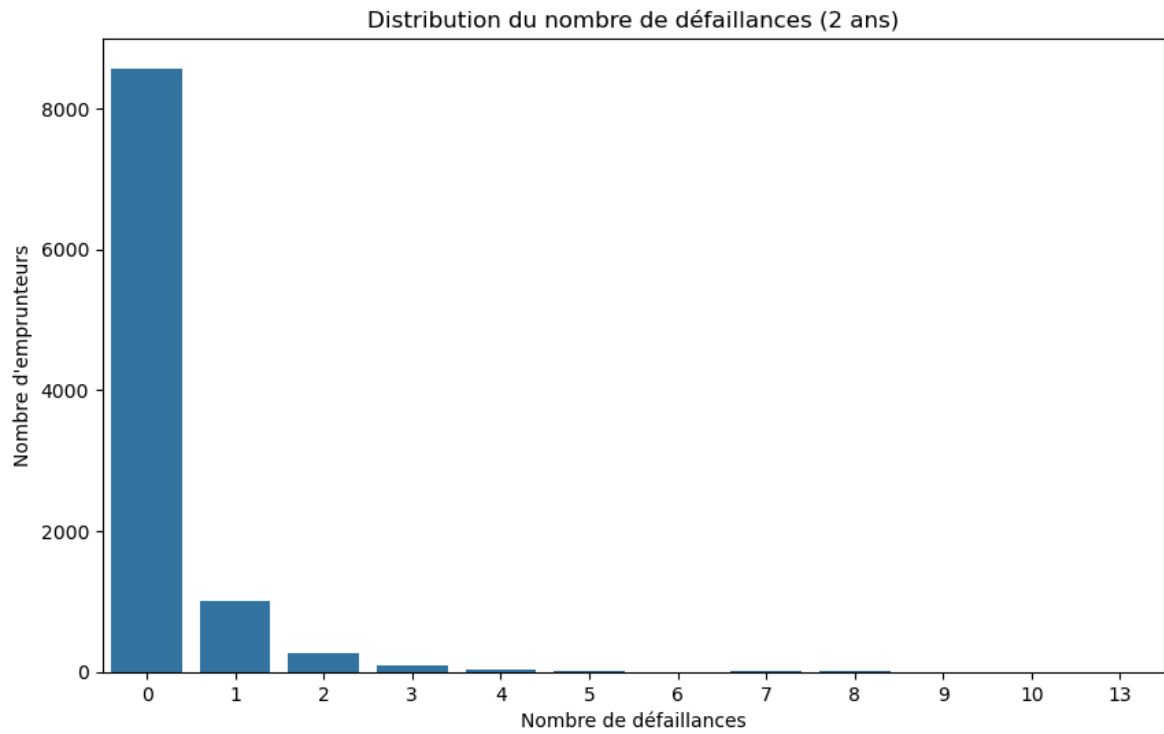
In [9]:

```

import matplotlib.pyplot as plt
import seaborn as sns

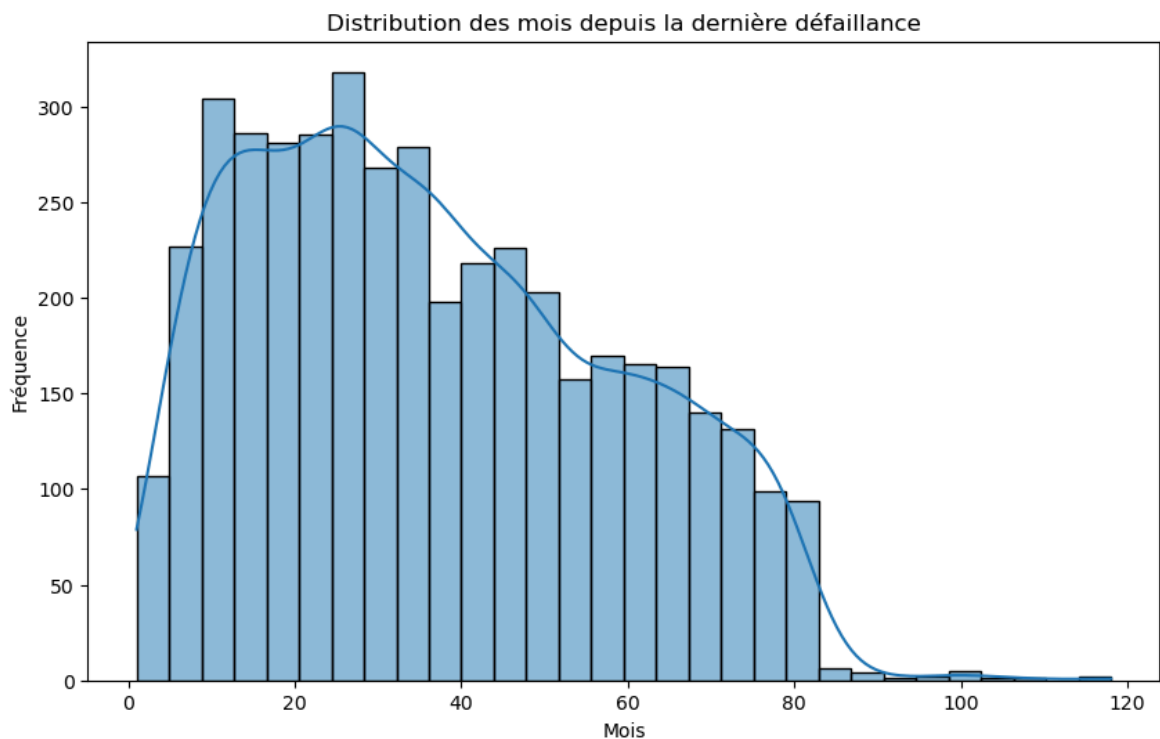
plt.figure(figsize=(10, 6))
sns.countplot(x='delinq_2y', data=df)
plt.title('Distribution du nombre de défaillances (2 ans)')
plt.xlabel('Nombre de défaillances')
plt.ylabel('Nombre d\'emprunteurs')
plt.show()

```

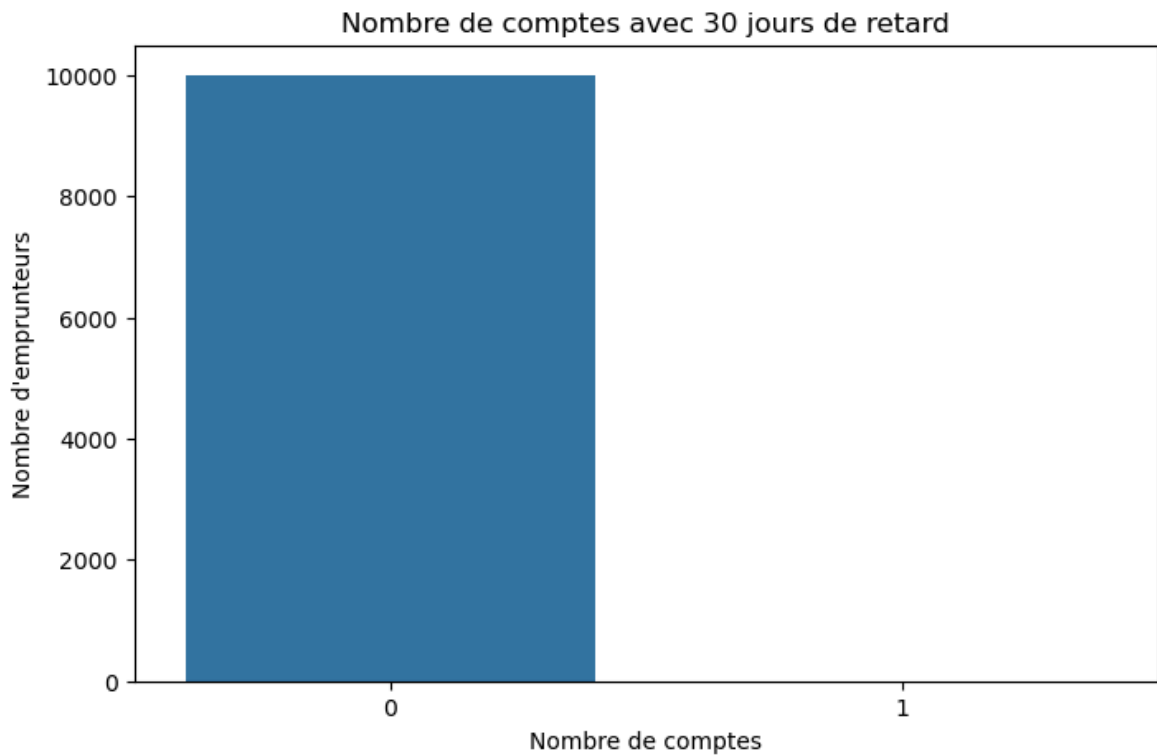
```
In [10]: df['months_since_last_delinq'] = pd.to_numeric(df['months_since_last_delinq'], e

plt.figure(figsize=(10, 6))
sns.histplot(df['months_since_last_delinq'].dropna(), bins=30, kde=True)
plt.title('Distribution des mois depuis la dernière défaillance')
plt.xlabel('Mois')
plt.ylabel('Fréquence')
plt.show()
```

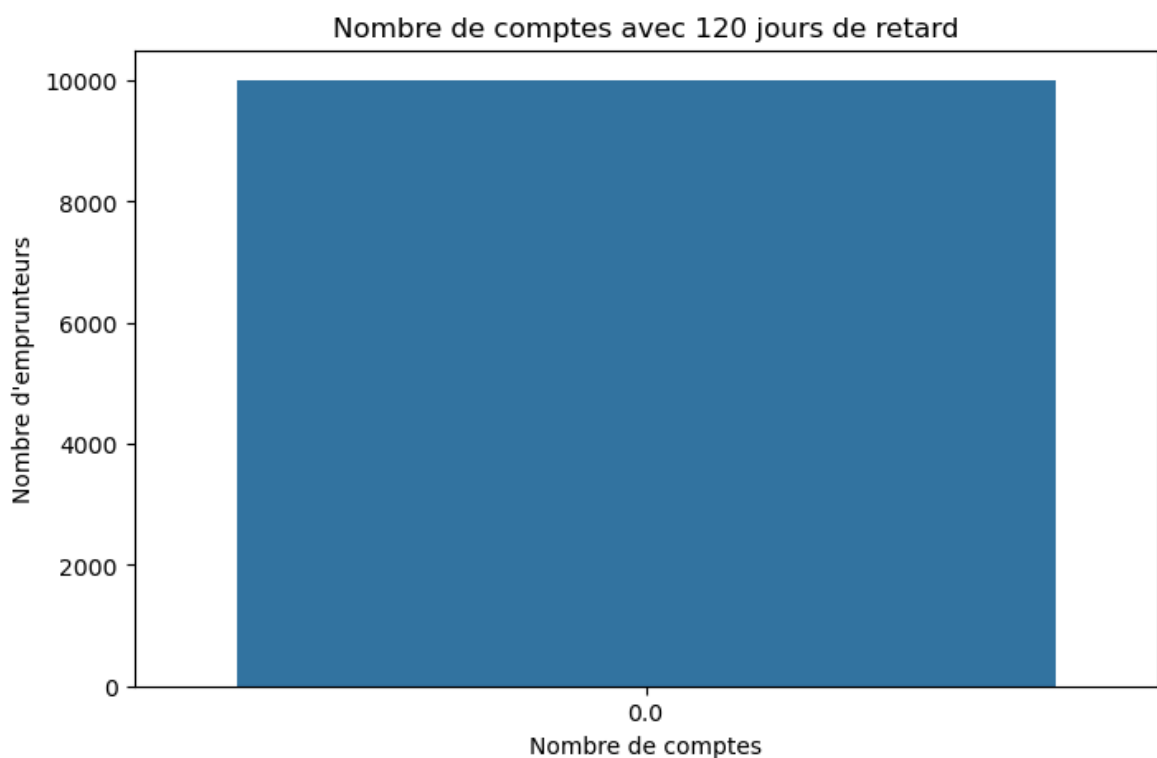


```
In [11]: plt.figure(figsize=(8, 5))
sns.countplot(x='num_accounts_30d_past_due', data=df)
plt.title('Nombre de comptes avec 30 jours de retard')
```

```
plt.xlabel('Nombre de comptes')
plt.ylabel('Nombre d\'emprunteurs')
plt.show()
```

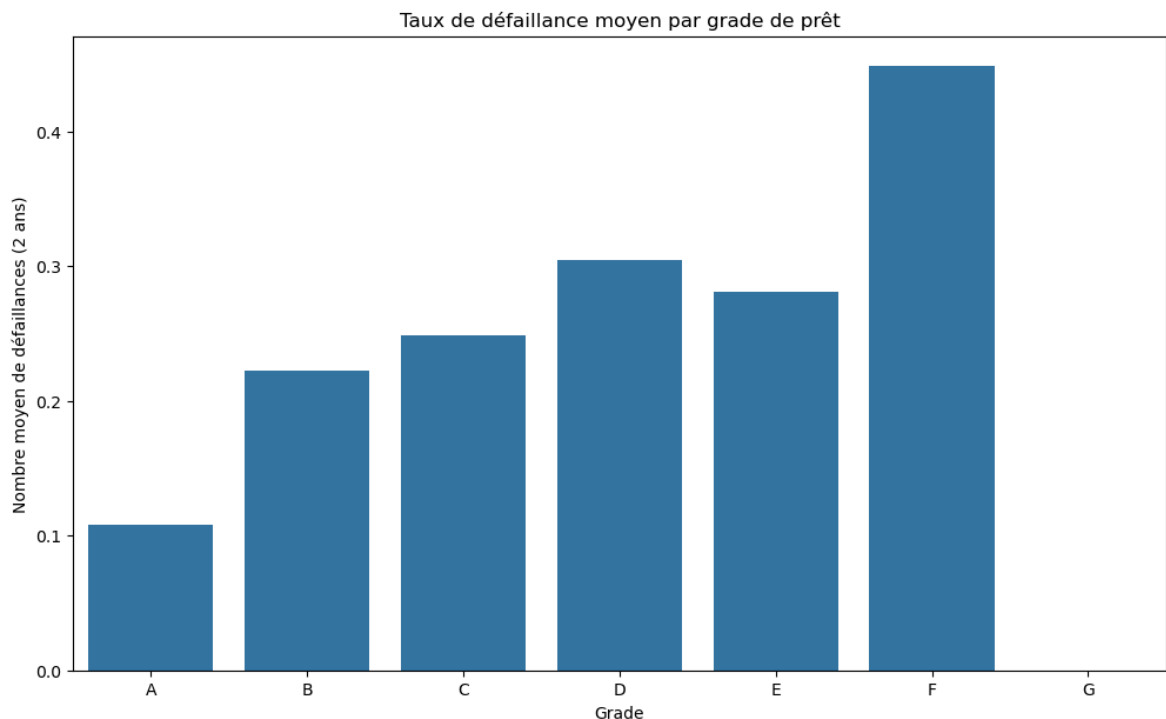


```
In [12]: plt.figure(figsize=(8, 5))
sns.countplot(x='num_accounts_120d_past_due', data=df)
plt.title('Nombre de comptes avec 120 jours de retard')
plt.xlabel('Nombre de comptes')
plt.ylabel('Nombre d\'emprunteurs')
plt.show()
```



```
In [13]: #création du taux de défaillance par grade
delinq_by_grade = df.groupby('grade')['delinq_2y'].mean().reset_index()
```

```
plt.figure(figsize=(12, 7))
sns.barplot(x='grade', y='delinq_2y', data=delinq_by_grade.sort_values('grade'))
plt.title('Taux de défaillance moyen par grade de prêt')
plt.xlabel('Grade')
plt.ylabel('Nombre moyen de défaillances (2 ans)')
plt.show()
```



2.2 Variables prédictives avancées

risque a 360 degre

Hormis les variables sociodémographiques ou financières de base, **le risque de défaillance d'un emprunteur** dépend parfois des phénomènes qui ne sont pas sous son contrôle. C'est ainsi que, afin de rendre notre modèle beaucoup plus sensible à l'environnement des emprunteurs, nous prendrons en compte des :

Variables basées sur les comportements passés d'un prêteur comme indicateur de son risque futur, car par expérience (d'après les experts), un client qui fait défaut une fois est susceptible de faire encore défaut. Ceci grâce aux variables comme **delinq_2y**, **months_since_last_delinq**, **num_accounts_past_due** qui renseignent sur l'historique de défauts, **account_stability**, **delinquency_rate** qui sont des indicateurs de discipline de paiement du client.

Variables **macroéconomiques** (taux de chômage, PIB,...) car si le PIB par habitant baisse, cela reflète une condition économique sévère pouvant avoir de l'impact sur la capacité de remboursement des clients. Dans le même ordre d'idée, si le taux de chômage augmente, la probabilité de défaillance des clients de ce pays augmente automatiquement.

Pareillement, la vie d'un pays ou d'une région est généralement animée par des **cycles économiques**. Il est question ici de prendre en compte tous ces paramètres car, on sait que certains secteurs ou métiers ont un revenu saisonnier (agriculture, tourisme, le transport, ...).

Bien évidemment, la meilleure manière de prévenir le risque est de le faire dans le temps. C'est ainsi que, pour prendre en compte cette dimension, il ne sera pas question pour nous de dire si le client risque de faire défaut ou non, mais d'introduire un paramètre qui nous permettra de dire de manière approximative quand le risque pourrait avoir lieu, afin de définir à temps la stratégie de recouvrement et aussi segmenter les clients selon leur horizon de risque.

```
In [14]: import pandas_datareader as web
import pandas as pd

# Liste des codes des États américains
state_codes = [
    'AL', 'AK', 'AZ', 'AR', 'CA', 'CO', 'CT', 'DE', 'FL', 'GA',
    'HI', 'ID', 'IL', 'IN', 'IA', 'KS', 'KY', 'LA', 'ME', 'MD',
    'MA', 'MI', 'MN', 'MS', 'MO', 'MT', 'NE', 'NV', 'NH', 'NJ',
    'NM', 'NY', 'NC', 'ND', 'OH', 'OK', 'OR', 'PA', 'RI', 'SC',
    'SD', 'TN', 'TX', 'UT', 'VT', 'VA', 'WA', 'WV', 'WI', 'WY'
]

# dates de début et de fin
start = "2000-01-01"
end = "2023-12-31"

all_states_unemployment = pd.DataFrame()

for state in state_codes:
    fred_code = f"{state}UR"
    try:
        # Téléchargez les données
        unemployment_data = web.DataReader(fred_code, "fred", start, end)
        unemployment_data = unemployment_data.rename(columns={fred_code: "unemployment_data"})
        unemployment_data["state"] = state

        # Concaténez les données
        all_states_unemployment = pd.concat([all_states_unemployment, unemployment_data])
    except Exception as e:
        print(f"Failed to fetch data for {state}: {e}")

# Affichez le résultat
all_states_unemployment
```

Out[14]:

	unemployment_rate	state
DATE		
2000-01-01	4.7	AL
2000-02-01	4.7	AL
2000-03-01	4.7	AL
2000-04-01	4.7	AL
2000-05-01	4.6	AL
...	...	...
2023-08-01	2.8	WY
2023-09-01	2.8	WY
2023-10-01	2.8	WY
2023-11-01	2.8	WY
2023-12-01	2.8	WY

14400 rows × 2 columns

```
!pip install pandas-datareader import datetime import requests import pandas_datareader.data as web # Définir la période start
= datetime.datetime(2000, 1, 1) end = datetime.datetime.today() # ===== # 1) TAUX DE CHÔMAGE
PAR ÉTAT (mensuel via FRED) # ===== # Exemple pour la Californie (CAUR) # Liste des codes
FRED : "CAUR", "TXUR", etc. (UR = unemployment rate) unemployment = web.DataReader("CAUR", "fred", start, end)
unemployment = unemployment.rename(columns={"CAUR": "unemployment_rate"}) unemployment["state"] = "CA"
unemployment # ===== # 3) INDICATEUR DE RÉCESSION (mensuel via FRED) #
===== recession = web.DataReader("USREC", "fred", start, end) recession =
recession.rename(columns={"USREC": "recession"}) recession.head() import pandas as pd # Essayer avec le séparateur point-
virgule pib_df = pd.read_csv("pib.csv", sep=";") # Si ça ne marche pas, essayer la tabulation # pib_df = pd.read_csv("pib.csv",
sep="\t") # Afficher un aperçu print(pib_df.head()) print(pib_df.columns) import pandas as pd # Charger en sautant les 3
premières lignes pib_df = pd.read_csv("pib.csv", skiprows=3) # Aperçu pib_df.head() #print(pib_df.columns)
print(df.columns.tolist())
```

In []:

In [15]: `df['issue_month'] = pd.to_datetime(df['issue_month'])`

```
# Ramener les deux colonnes au niveau "mois"
df['issue_month'] = df['issue_month'].dt.to_period('M').dt.to_timestamp()
```

C:\Users\USER\AppData\Local\Temp\ipykernel_14616\4141016630.py:1: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.

```
df['issue_month'] = pd.to_datetime(df['issue_month'])
```

```
In [16]: unemployment_2018 = (
    all_states_unemployment.reset_index()
)
unemployment_2018['DATE'] = unemployment_2018['DATE'].dt.to_period('M').dt.to_timestamp()

# Garder seulement 2018
unemployment_2018 = unemployment_2018[unemployment_2018['DATE'].dt.year == 2018]

# Fusion (clé = issue_month et state)
```

```
df = df.merge(
    unemployment_2018.rename(columns={'DATE': 'issue_month'}),
    on=['issue_month', 'state'],
    how='left'
)

df.head()
```

Out[16]:

	emp_title	emp_length	state	homeownership	annual_income	verified_income	deb
--	-----------	------------	-------	---------------	---------------	-----------------	-----

0	global config engineer	3.0	NJ	MORTGAGE	90000.0	Verified	
1	warehouse office clerk	10.0	HI	RENT	40000.0	Not Verified	
2	assembly	3.0	WI	RENT	40000.0	Source Verified	
3	customer service	1.0	PA	RENT	30000.0	Not Verified	
4	security supervisor	10.0	CA	RENT	35000.0	Verified	

5 rows × 62 columns



df = pd.merge(df, recession[['DATE', 'recession']], # adapte le nom de la colonne indiquant la récession left_on='date', right_on='DATE', how='left')

In [17]:

```
df['issue_month']
#•      issue_month : Mois d'émission du prêt.
```

Out[17]:

```
0      2018-03-01
1      2018-02-01
2      2018-02-01
3      2018-01-01
4      2018-03-01
...
9995   2018-01-01
9996   2018-02-01
9997   2018-02-01
9998   2018-02-01
9999   2018-02-01
Name: issue_month, Length: 10000, dtype: datetime64[ns]
```

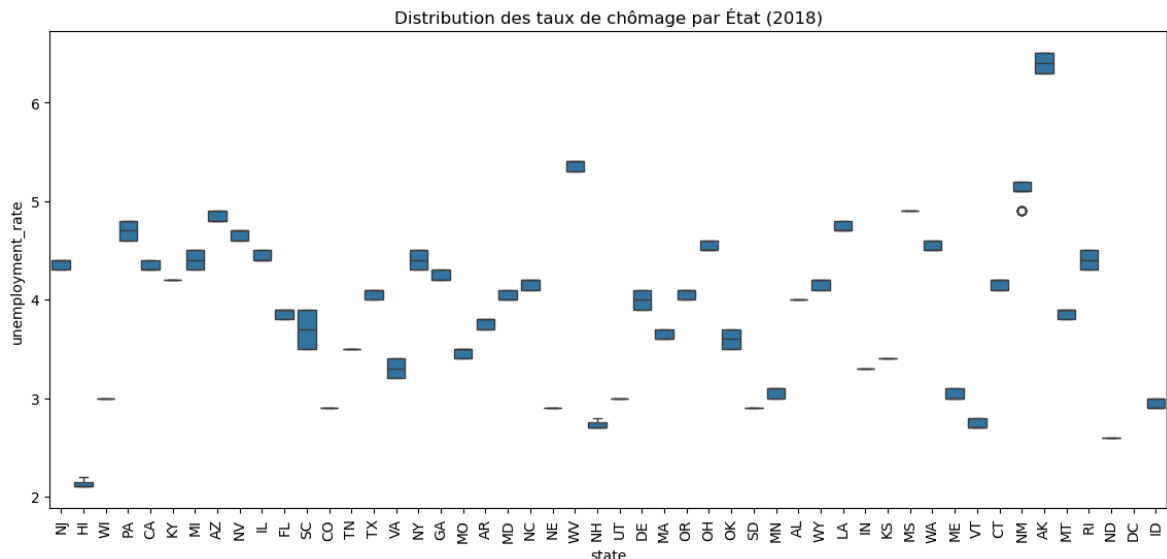
Dans notre df la date d'emission des prets c'est 2018. afin de maintenir une coherence entre les taux de chaumage et les et les autre informations, nous nous somme concentrer sur ls taux de chomage par etat en 2018.

In [18]:

```
import seaborn as sns

plt.figure(figsize=(14,6))
sns.boxplot(x="state", y="unemployment_rate", data=df)
plt.xticks(rotation=90)
```

```
plt.title("Distribution des taux de chômage par État (2018)")
plt.show()
```



2.3 Traitement des segments du dataset

Applications Jointes

L'objectif ici est de mener une étude comparative entre les applications jointes et les applications individuelles. Pour le faire, nous créons une variable que nous appelons **is_joint_application** qui marque une application jointe si **annual_income_joint** est > 0 car initialement nous avons remplacé les valeurs manquantes des variables jointes par 0 du fait de la grande proportion de valeurs manquantes qu'on avait pour ces variables. Une telle étude nous permet de voir la différence entre les deux groupes suivant plusieurs variables, et de voir si un groupe augmente le risque de défaut par rapport à l'autre.

```
In [19]: def analyse_applications_jointes(df):
    """
    Analyse des applications jointes avec les variables spécifiques.
    Compatible même si les valeurs manquantes ont été remplacées par 0.
    """
    print("=== ANALYSE DES APPLICATIONS JOINTES ===")

    # --- Création du flag d'application conjointe ---
    # Si annual_income_joint est > 0, alors c'est une application conjointe
    df['is_joint_application'] = df['annual_income_joint'] > 0

    # --- Statistiques de base ---
    nb_joint = df['is_joint_application'].sum()
    pourcentage_joint = df['is_joint_application'].mean() * 100
    print(f"Nombre d'applications jointes: {nb_joint}")
    print(f"Pourcentage d'applications jointes: {pourcentage_joint:.2f}%")

    # --- Analyse comparative ---
    colonnes_dispo = [col for col in [
        'annual_income', 'annual_income_joint',
        'debt_to_income', 'debt_to_income_joint',
        'loan_amount', 'interest_rate'
```

```

] if col in df.columns]

joint_stats = df.groupby('is_joint_application')[colonnes_dispo].agg(['mean'])

print("\nComparatif Applications Individuelles vs Jointes:")
print(joint_stats)

# --- Analyse de la vérification des revenus ---
if 'verification_income_joint' in df.columns:
    verification_stats = pd.crosstab(
        df['is_joint_application'],
        df['verification_income_joint'],
        normalize='index'
    ).round(4)
    print("\nDistribution de la vérification des revenus:")
    print(verification_stats)

# --- Visualisation ---
fig, axes = plt.subplots(2, 3, figsize=(18, 12))

# Revenu total comparé
df['total_income_joint'] = df['annual_income'] + df['annual_income_joint']
df.boxplot(column='total_income_joint', by='is_joint_application', ax=axes[0,0])
axes[0,0].set_title("Revenu Total par Type d'Application")
axes[0,0].set_ylabel('Revenu Total ($)')

# DTI comparé
if 'debt_to_income' in df.columns:
    df.boxplot(column='debt_to_income', by='is_joint_application', ax=axes[0,1])
    axes[0,1].set_title('DTI par Type d\'Application')
    axes[0,1].set_ylabel('Debt-to-Income Ratio')

# Montant du prêt
if 'loan_amount' in df.columns:
    df.boxplot(column='loan_amount', by='is_joint_application', ax=axes[0,2])
    axes[0,2].set_title('Montant du Prêt par Type d\'Application')
    axes[0,2].set_ylabel('Montant du Prêt ($)')

# Taux d'intérêt
if 'interest_rate' in df.columns:
    df.boxplot(column='interest_rate', by='is_joint_application', ax=axes[1,0])
    axes[1,0].set_title('Taux d\'Intérêt par Type d\'Application')
    axes[1,0].set_ylabel('Taux d\'Intérêt (%)')

# Vérification des revenus (si disponible)
if 'verification_income_joint' in df.columns:
    verification_counts = df[df['is_joint_application']]['verification_income_joint'].value_counts()
    axes[1,1].pie(verification_counts.values, labels=verification_counts.index)
    axes[1,1].set_title('Vérification Revenus - Applications Jointes')

# Ratio DTI joint
if 'debt_to_income_joint' in df.columns:
    df.boxplot(column='debt_to_income_joint', by='is_joint_application', ax=axes[1,2])
    axes[1,2].set_title('DTI Joint par Type d\'Application')
    axes[1,2].set_ylabel('DTI Joint')

plt.suptitle('')
plt.tight_layout()
plt.show()

```



```
return df
```

```
In [20]: analyse_applications_jointes(df)
```

```
=== ANALYSE DES APPLICATIONS JOINTES ===
```

```
Nombre d'applications jointes: 1495
```

```
Pourcentage d'applications jointes: 14.95%
```

```
Comparatif Applications Individuelles vs Jointes:
```

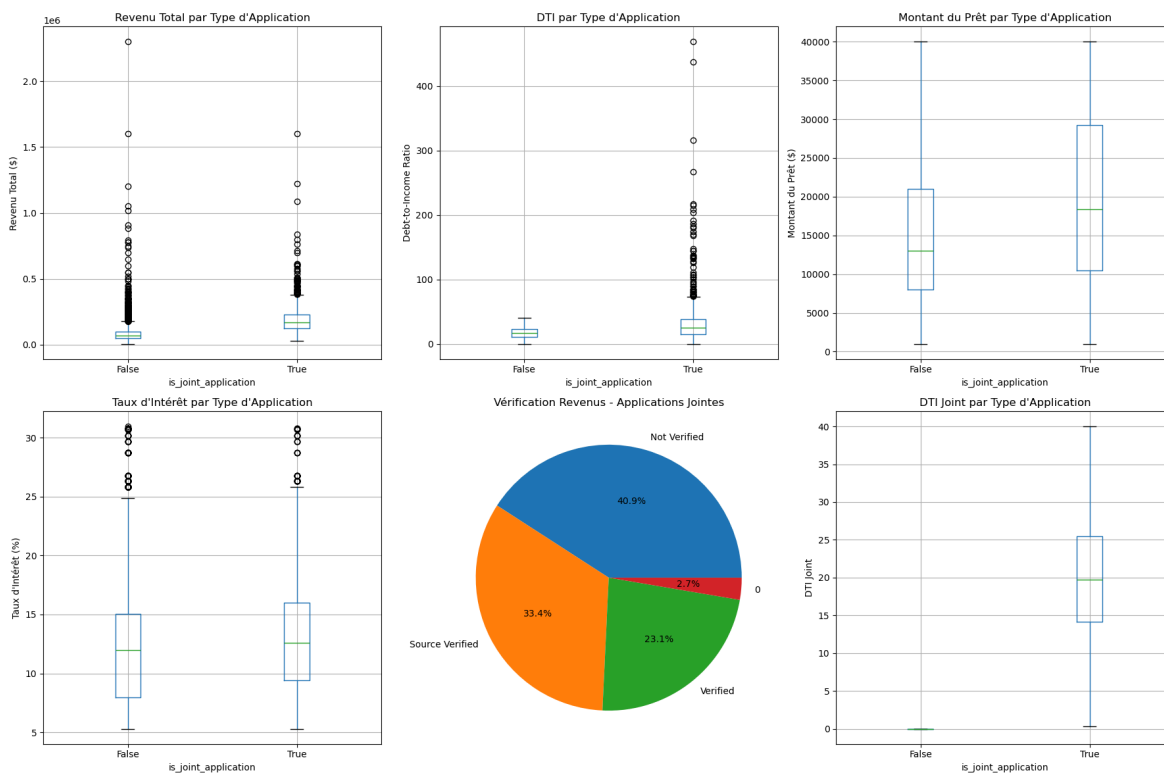
	annual_income		annual_income_joint	
	mean	median	mean	median
is_joint_application				
False	82321.92	68200.0	0.00	0.0
True	61587.68	53000.0	127914.57	113000.0

	debt_to_income		debt_to_income_joint	
	mean	median	mean	median
is_joint_application				
False	17.34	16.79	0.00	0.00
True	30.51	25.23	19.98	19.72

	loan_amount		interest_rate	
	mean	median	mean	median
is_joint_application				
False	15748.84	13000.0	12.30	11.98
True	19849.70	18400.0	13.16	12.61

```
Distribution de la vérification des revenus:
```

	0	Not Verified	Source Verified	Verified
is_joint_application				
False	1.0000	0.0000	0.0000	0.0000
True	0.0268	0.4087	0.3338	0.2308



Out[20]:

	emp_title	emp_length	state	homeownership	annual_income	verified_income
0	global config engineer	3.0	NJ	MORTGAGE	90000.0	Verified
1	warehouse office clerk	10.0	HI	RENT	40000.0	Not Verified
2	assembly	3.0	WI	RENT	40000.0	Source Verified
3	customer service	1.0	PA	RENT	30000.0	Not Verified
4	security supervisor	10.0	CA	RENT	35000.0	Verified
...	...	...	...	...	...	...
9995	owner	10.0	TX	RENT	108000.0	Source Verified
9996	director	8.0	PA	MORTGAGE	121000.0	Verified
9997	toolmaker	10.0	CT	MORTGAGE	67000.0	Verified
9998	manager	1.0	WI	MORTGAGE	80000.0	Source Verified
9999	operations analyst	3.0	CT	RENT	66000.0	Not Verified

10000 rows × 64 columns



On observe que 14,95 % des demandes de prêt sont des applications conjointes. Bien que cette proportion soit relativement faible, elle n'est pas négligeable. En outre, le fait qu'elle soit minoritaire pourrait indiquer des différences notables dans la situation financière de ces ménages par rapport aux emprunteurs individuels.

Nous constatons également que les emprunteurs individuels ont un revenu annuel moyen individuel plus élevé (environ 82 322\$) que le revenu moyen individuel des demandeurs dans les applications conjointes (environ 61 588 dollar).

Cependant, en combinant le revenu individuel du co-emprunteur au revenu conjoint rapporté sur la demande ($61\,588 + 127\,915 \approx 189\,503$ \$), on obtient un revenu annuel agrégé nettement plus important. Ce niveau de revenu élevé implique que les applications conjointes possèdent une grande capacité de remboursement et présentent, de ce fait, une crédibilité financière accrue pour la banque.

Dans le même ordre d'idée, l'analyse du ratio d'endettement (DTI, Debt-to-Income) révèle des tendances intéressantes :

Les applications individuelles affichent un DTI moyen d'environ 17 %.

Les demandeurs dans les applications conjointes ont un DTI individuel moyen d'environ 30,5 %, mais leur DTI combiné est nettement plus faible, avoisinant les 20 %.

Le fait que le DTI combiné soit inférieur au DTI du demandeur pris isolément témoigne d'une meilleure maîtrise de la gestion de crédit et d'une charge de dette plus soutenable lorsque les revenus des deux parties sont pris en compte.

Enfin, il est constaté que les applications conjointes reçoivent en moyenne des montants de prêt plus importants (moyenne de 19 849,70 \$) que les applications individuelles (moyenne de 15 748,84 dollar). De plus, environ 75 % des prêts conjoints reçoivent des montants allant jusqu'à 290 000 dollar.

Cette tendance est logique : puisque les applications conjointes déclarent un revenu annuel agrégé plus élevé, elles ont accès à des sommes plus importantes. Toutefois, le fait que ces prêts plus élevés s'accompagnent souvent d'un taux d'intérêt légèrement supérieur peut s'expliquer par l'augmentation du risque absolu (la somme potentielle perdue) que représente l'octroi d'un prêt d'un montant aussi grand, même si le risque relatif (DTI) est bien géré.

analyse_segmentation_objectif

L'objectif ici est de comprendre les profils de prêts selon la finalité des prêts. Pour cela, on compte le nombre d'occurrences de chaque type d'objectif et, par la suite, on affiche le nombre et le pourcentage de prêts par objectif. L'idée c'est de voir **quel type d'objectif domine le portefeuille**. Et enfin, on identifie les segments où les prêts sont les plus élevés, et les taux les plus forts.

```
In [21]: def analyse_segmentation_objectif(df):
        """
        Analyse par objectif du crédit (loan_purpose)
        """
        print("\n=== SEGMENTATION PAR OBJECTIF DU CRÉDIT ===")

        # Distribution des objectifs
        purpose_distribution = df['loan_purpose'].value_counts()
        print("Distribution des objectifs de crédit:")
        for purpose, count in purpose_distribution.items():
            pourcentage = (count / len(df)) * 100
            print(f" {purpose}: {count} ({pourcentage:.1f}%)")

        # Analyse détaillée par objectif
        purpose_analysis = df.groupby('loan_purpose').agg({
            'loan_amount': ['count', 'mean', 'median', 'std'],
            'annual_income': ['mean', 'median'],
            'debt_to_income': ['mean', 'median'],
            'interest_rate': ['mean', 'median'],
            'installment': ['mean', 'median']
        }).round(2)

        print("\nAnalyse détaillée par objectif:")
        print(purpose_analysis)

        # Variables de risque par objectif
        risk_metrics = [
            'delinq_2y', 'num_historical_failed_to_pay', 'current_accounts_delinq',
            'num_collections_last_12m', 'account_never_delinq_percent'
```

```

]

available_risk_metrics = [metric for metric in risk_metrics if metric in df.

if available_risk_metrics:
    risk_analysis = df.groupby('loan_purpose')[available_risk_metrics].mean()
    print("\nMétriques de risque par objectif:")
    print(risk_analysis)

# Visualisation
fig, axes = plt.subplots(2, 3, figsize=(20, 12))

# Top 10 objectifs
top_purposes = purpose_distribution.head(10)
axes[0,0].barh(range(len(top_purposes)), top_purposes.values)
axes[0,0].set_yticks(range(len(top_purposes)))
axes[0,0].set_yticklabels(top_purposes.index)
axes[0,0].set_title('Top 10 Objectifs de Crédit')
axes[0,0].set_xlabel('Nombre de Demandes')

# Montant moyen par objectif
avg_loan_by_purpose = df.groupby('loan_purpose')['loan_amount'].mean().sort_
axes[0,1].barh(range(len(avg_loan_by_purpose)), avg_loan_by_purpose.values)
axes[0,1].set_yticks(range(len(avg_loan_by_purpose)))
axes[0,1].set_yticklabels(avg_loan_by_purpose.index)
axes[0,1].set_title('Top 10 - Montant Moyen par Objectif')
axes[0,1].set_xlabel('Montant Moyen ($)')

# Taux d'intérêt moyen par objectif
avg_interest_by_purpose = df.groupby('loan_purpose')['interest_rate'].mean()
axes[0,2].barh(range(len(avg_interest_by_purpose)), avg_interest_by_purpose.
axes[0,2].set_yticks(range(len(avg_interest_by_purpose)))
axes[0,2].set_yticklabels(avg_interest_by_purpose.index)
axes[0,2].set_title('Top 10 - Taux d\'Intérêt Moyen')
axes[0,2].set_xlabel('Taux d\'Intérêt Moyen (%)')

# DTI moyen par objectif
avg_dti_by_purpose = df.groupby('loan_purpose')['debt_to_income'].mean().sor
axes[1,0].barh(range(len(avg_dti_by_purpose)), avg_dti_by_purpose.values)
axes[1,0].set_yticks(range(len(avg_dti_by_purpose)))
axes[1,0].set_yticklabels(avg_dti_by_purpose.index)
axes[1,0].set_title('Top 10 - DTI Moyen par Objectif')
axes[1,0].set_xlabel('DTI Moyen')

# Revenu moyen par objectif
avg_income_by_purpose = df.groupby('loan_purpose')['annual_income'].mean().s
axes[1,1].barh(range(len(avg_income_by_purpose)), avg_income_by_purpose.valu
axes[1,1].set_yticks(range(len(avg_income_by_purpose)))
axes[1,1].set_yticklabels(avg_income_by_purpose.index)
axes[1,1].set_title('Top 10 - Revenu Moyen par Objectif')
axes[1,1].set_xlabel('Revenu Moyen ($)')

# Délinquance par objectif
if 'delinq_2y' in df.columns:
    avg_delinq_by_purpose = df.groupby('loan_purpose')['delinq_2y'].mean().s
    axes[1,2].barh(range(len(avg_delinq_by_purpose)), avg_delinq_by_purpose.
    axes[1,2].set_yticks(range(len(avg_delinq_by_purpose)))
    axes[1,2].set_yticklabels(avg_delinq_by_purpose.index)
    axes[1,2].set_title('Top 10 - Délinquance Moyenne (2 ans)')
    axes[1,2].set_xlabel('Délinquances Moyennes')

```

```
plt.tight_layout()  
plt.show()  
  
return df
```

On observe que debt_consolidation (51,4 %) et credit_card (22,5 %) représentent plus de 74 % des motifs de prêt. Ce qui signifie que plus de 74 % des prêts sont destinés au recouvrement d'autres dettes. Ce fait peut être interprété comme un signe de risque de crédit élevé. Ces deux catégories se présentent comme les premiers motifs d'application.

Dans le même ordre d'idée, other (9,1 %), home_improvement (6,8 %) et major_purchase (3,03 %) représentent environ 18 % des motifs, qui sont des motifs secondaires, liés à l'amélioration de l'habitat et aux achats. Les autres, avec une proportion faible, environ 8 %, sont liés à la consommation courante ou spécialisée. On observe donc ici que très peu d'applications, pour ne pas dire presque aucune, n'a pour motif l'investissement. Ce qui représente un très grand risque de défaut de crédit et peut témoigner d'une santé financière très fragile.

On observe aussi qu'en moyenne, les prêts pour les petites entreprises sont les plus élevés (Small Business (19 820 \$)) que les autres. De même, les emprunteurs ayant les revenus moyens les plus élevés (pour major_purchase, home_improvement) ne sont pas ceux qui contractent les prêts les plus élevés, ce qui peut impliquer qu'ils utilisent leurs fonds propres ou que les projets financés restent d'une taille modérée.

Malgré son volume (51,4 % des cas), l'objectif de consolidation de dette a un prêt moyen élevé (17 645 \$), signalant que les emprunteurs cherchent à regrouper des montants importants.

Le DTI moyen le plus élevé est observé pour la consolidation de dette (20,42 %) et les cartes de crédit (19,81 %). Cela confirme que ces emprunteurs sont déjà lourdement endettés, ce qui justifie leur demande de consolidation.

Le DTI le plus bas est pour l'achat de maison (12,61 %), ce qui indique une gestion de dette plus saine pour ce type de demande. Les taux d'intérêt moyens les plus élevés sont attribués aux objectifs les moins fréquents et possiblement perçus comme plus risqués : Renewable Energy (13,54 %), Vacation (13,33 %) et Moving (13,17 %).

Les objectifs de consolidation et de petite entreprise ont également des taux élevés, suggérant un risque perçu important par le prêteur.

Les objectifs les moins chers sont House (11,37 %) et Credit Card (11,43 %), potentiellement en raison d'une garantie implicite (maison) ou d'un profil moins risqué pour le remboursement des cartes. Les prêts pour Vacances (0,4677) et Home Improvement (0,3206) montrent la délinquance la plus élevée au cours des 2 dernières années, suggérant que ces emprunteurs ont eu plus de difficultés de paiement. C'est contre-intuitif pour Home Improvement, qui était associé à un revenu élevé.

Les prêts pour l'achat de Maison (0,1457) et l'Énergie Renouvelable (0,1000) affichent le taux de délinquance le plus faible, confirmant leur profil de risque prudent. La

consolidation de dette (0,1841) présente le plus grand nombre moyen d'échecs de paiement, renforçant l'idée que ces emprunteurs ont un historique de gestion de crédit plus difficile.

```
In [22]: analyse_segmentation_objectif(df)
```

=== SEGMENTATION PAR OBJECTIF DU CRÉDIT ===

Distribution des objectifs de crédit:

debt_consolidation: 5144 (51.4%)
 credit_card: 2249 (22.5%)
 other: 914 (9.1%)
 home_improvement: 680 (6.8%)
 major_purchase: 303 (3.0%)
 medical: 162 (1.6%)
 house: 151 (1.5%)
 car: 131 (1.3%)
 small_business: 125 (1.2%)
 moving: 69 (0.7%)
 vacation: 62 (0.6%)
 renewable_energy: 10 (0.1%)

Analyse détaillée par objectif:

loan_purpose	loan_amount				annual_income \	
	count	mean	median	std	mean	
car	131	10391.22	8000.0	8015.61	66148.56	
credit_card	2249	15540.78	13000.0	9680.76	76261.03	
debt_consolidation	5144	17645.40	15000.0	10085.02	78681.84	
home_improvement	680	17234.15	15000.0	11101.20	93426.14	
house	151	17809.77	15000.0	11349.89	80608.51	
major_purchase	303	15684.57	12000.0	11296.33	98784.70	
medical	162	11338.27	8000.0	9366.59	77915.86	
moving	69	9893.84	6000.0	9107.90	71935.12	
other	914	12931.10	10000.0	10330.92	73764.01	
renewable_energy	10	14000.00	10000.0	11635.67	96750.00	
small_business	125	19820.00	17950.0	11817.86	90723.96	
vacation	62	6804.84	5200.0	4477.17	70289.22	

loan_purpose	debt_to_income		interest_rate		\
	median	mean	median	mean	
car	60000.0	15.61	14.27	12.04	10.90
credit_card	65000.0	19.81	18.42	11.43	10.90
debt_consolidation	65000.0	20.42	18.37	13.01	11.99
home_improvement	75000.0	18.41	16.36	11.62	10.90
house	71487.0	12.61	11.84	11.37	10.42
major_purchase	66000.0	14.21	12.14	11.97	10.90
medical	65000.0	17.89	15.44	12.15	10.90
moving	60000.0	17.07	17.49	13.17	12.61
other	60000.0	16.58	15.12	12.43	11.98
renewable_energy	66500.0	18.67	18.15	13.54	12.30
small_business	75000.0	16.68	15.95	13.00	12.61
vacation	59279.5	19.50	19.23	13.33	13.58

loan_purpose	installment	
	mean	median
car	303.25	231.67
credit_card	458.23	371.64
debt_consolidation	512.50	451.70
home_improvement	490.99	398.88
house	498.75	433.86
major_purchase	454.79	361.38
medical	335.06	248.36
moving	284.10	193.38
other	377.66	302.23

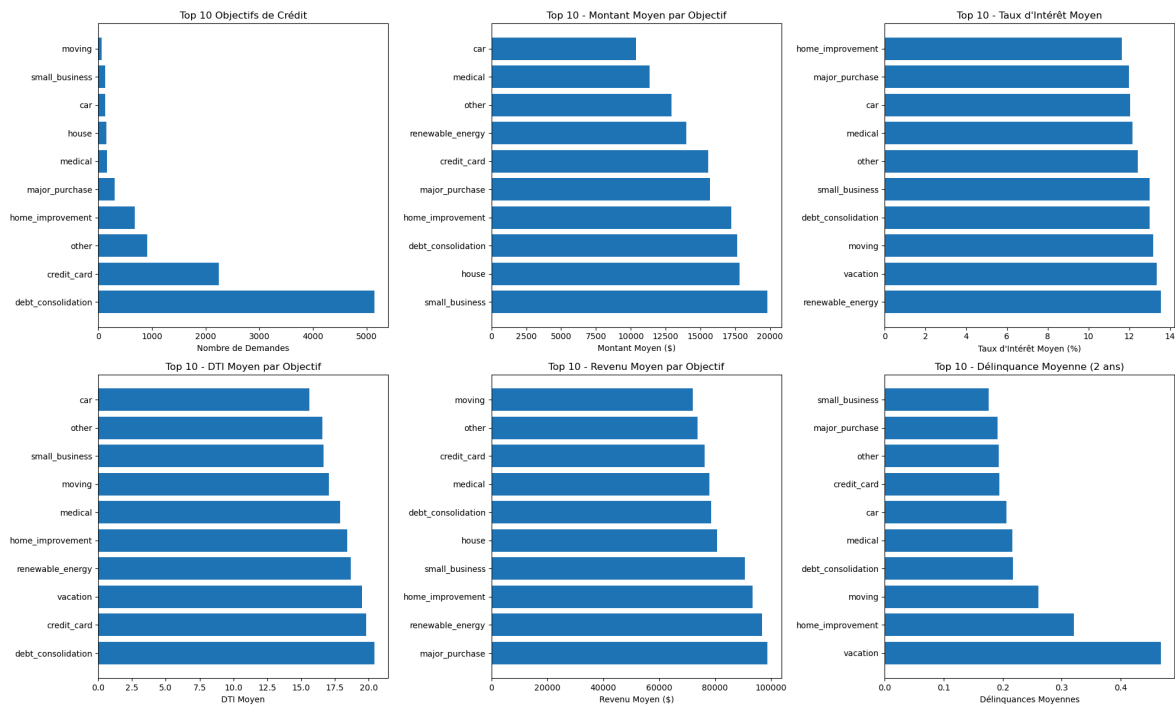
renewable_energy	408.92	314.80
small_business	574.96	511.39
vacation	217.37	190.06

Métriques de risque par objectif:

	delinq_2y	num_historical_failed_to_pay \
loan_purpose		
car	0.2061	0.1603
credit_card	0.1943	0.1436
debt_consolidation	0.2170	0.1841
home_improvement	0.3206	0.1809
house	0.1457	0.1457
major_purchase	0.1914	0.1122
medical	0.2160	0.1667
moving	0.2609	0.1304
other	0.1937	0.1510
renewable_energy	0.1000	0.2000
small_business	0.1760	0.1360
vacation	0.4677	0.1290

	current_accounts_delinq	num_collections_last_12m \
loan_purpose		
car	0.0000	0.0000
credit_card	0.0000	0.0124
debt_consolidation	0.0000	0.0144
home_improvement	0.0000	0.0147
house	0.0000	0.0132
major_purchase	0.0000	0.0000
medical	0.0000	0.0000
moving	0.0000	0.0435
other	0.0011	0.0208
renewable_energy	0.0000	0.0000
small_business	0.0000	0.0080
vacation	0.0000	0.0161

	account_never_delinq_percent
loan_purpose	
car	94.6885
credit_card	95.3345
debt_consolidation	94.6707
home_improvement	94.0085
house	93.1907
major_purchase	94.3739
medical	94.1198
moving	94.6812
other	94.0414
renewable_energy	98.5700
small_business	92.5120
vacation	93.7968



Out[22]:

	emp_title	emp_length	state	homeownership	annual_income	verified_income
0	global config engineer	3.0	NJ	MORTGAGE	90000.0	Verified
1	warehouse office clerk	10.0	HI	RENT	40000.0	Not Verified
2	assembly	3.0	WI	RENT	40000.0	Source Verified
3	customer service	1.0	PA	RENT	30000.0	Not Verified
4	security supervisor	10.0	CA	RENT	35000.0	Verified
...	...	...	...	...	...	...
9995	owner	10.0	TX	RENT	108000.0	Source Verified
9996	director	8.0	PA	MORTGAGE	121000.0	Verified
9997	toolmaker	10.0	CT	MORTGAGE	67000.0	Verified
9998	manager	1.0	WI	MORTGAGE	80000.0	Source Verified
9999	operations analyst	3.0	CT	RENT	66000.0	Not Verified

10000 rows × 64 columns



On sait qu'en scoring de crédit, les risques de crédit peuvent varier en fonction des zones géographiques. Il est donc question ici de regrouper les données par État (groupby('state')) puis de calculer les statistiques de base pour chaque variable afin de voir quels sont les États où on a plus d'applications et quels sont les risques y afférents.

Par la suite, on trie les États par nombre de prêts (count) et on affiche les 10 plus importants. Cela donne une vision des marchés les plus actifs.

analyse_segmentation_geographique

```
In [23]: def analyse_segmentation_geographique(df):
        """
        Analyse géographique par état
        """
        print("\n=== SEGMENTATION GÉOGRAPHIQUE PAR ÉTAT ===")

        # Analyse de base par état
        state_analysis = df.groupby('state').agg({
            'loan_amount': ['count', 'mean', 'median'],
            'annual_income': ['mean', 'median'],
            'debt_to_income': ['mean', 'median'],
            'interest_rate': ['mean', 'median'],
            'unemployment_rate': 'mean' if 'unemployment_rate' in df.columns else 'c
        }).round(2)

        state_analysis.columns = ['count', 'loan_mean', 'loan_median', 'income_mean',
                                'dti_mean', 'dti_median', 'interest_mean', 'interest

        print("Top 10 états par volume de prêts:")
        print(state_analysis.nlargest(10, 'count'))

        # Analyse des variables de risque par état
        risk_metrics = [
            'delinq_2y', 'num_historical_failed_to_pay', 'current_accounts_delinq',
            'num_collections_last_12m', 'account_never_delinq_percent'
        ]

        available_risk_metrics = [metric for metric in risk_metrics if metric in df.

        if available_risk_metrics:
            state_risk_analysis = df.groupby('state')[available_risk_metrics].mean()
            print("\nMétriques de risque par état:")
            print(state_risk_analysis.nlargest(10, available_risk_metrics[0]))

        # Visualisation
        fig, axes = plt.subplots(2, 3, figsize=(10, 8))

        # Volume par état (top 15)
        top_states_volume = state_analysis.nlargest(15, 'count')
        axes[0,0].bar(range(len(top_states_volume)), top_states_volume['count'])
        axes[0,0].set_title('Top 15 États - Volume de Prêts')
        axes[0,0].set_ylabel('Nombre de Prêts')
        axes[0,0].set_xticks(range(len(top_states_volume)))
        axes[0,0].set_xticklabels(top_states_volume.index, rotation=45, ha='right')

        # Revenu moyen par état
        top_income_states = state_analysis.nlargest(15, 'income_mean')
        axes[0,1].bar(range(len(top_income_states)), top_income_states['income_mean'])
        axes[0,1].set_title('Top 15 États - Revenu Moyen')
        axes[0,1].set_ylabel('Revenu Moyen ($)')
        axes[0,1].set_xticks(range(len(top_income_states)))
        axes[0,1].set_xticklabels(top_income_states.index, rotation=45, ha='right')
```

```

# Taux d'intérêt moyen par état
top_interest_states = state_analysis.nlargest(15, 'interest_mean')
axes[0,2].bar(range(len(top_interest_states)), top_interest_states['interest_mean'])
axes[0,2].set_title('Top 15 États - Taux d\'Intérêt Moyen')
axes[0,2].set_ylabel('Taux d\'Intérêt Moyen (%)')
axes[0,2].set_xticks(range(len(top_interest_states)))
axes[0,2].set_xticklabels(top_interest_states.index, rotation=45, ha='right')

# DTI moyen par état
top_dti_states = state_analysis.nlargest(15, 'dti_mean')
axes[1,0].bar(range(len(top_dti_states)), top_dti_states['dti_mean'])
axes[1,0].set_title('Top 15 États - DTI Moyen')
axes[1,0].set_ylabel('DTI Moyen')
axes[1,0].set_xticks(range(len(top_dti_states)))
axes[1,0].set_xticklabels(top_dti_states.index, rotation=45, ha='right')

# Délinquance par état
if 'delinq_2y' in df.columns:
    delinq_by_state = df.groupby('state')['delinq_2y'].mean().nlargest(15)
    axes[1,1].bar(range(len(delinq_by_state)), delinq_by_state.values)
    axes[1,1].set_title('Top 15 États - Délinquance Moyenne')
    axes[1,1].set_ylabel('Délinquances Moyennes (2 ans)')
    axes[1,1].set_xticks(range(len(delinq_by_state)))
    axes[1,1].set_xticklabels(delinq_by_state.index, rotation=45, ha='right')

# Chômage vs Revenu
if 'unemployment_rate' in df.columns:
    scatter_data = df.groupby('state').agg({
        'annual_income': 'mean',
        'unemployment_rate': 'mean'
    }).dropna()

    if len(scatter_data) > 5:
        axes[1,2].scatter(scatter_data['unemployment_rate'], scatter_data['annual_income'])
        axes[1,2].set_xlabel('Taux de Chômage Moyen (%)')
        axes[1,2].set_ylabel('Revenu Annuel Moyen ($)')
        axes[1,2].set_title('Chômage vs Revenu par État')

        # Annoter quelques points
        for state in scatter_data.index[:5]:
            axes[1,2].annotate(state,
                               (scatter_data.loc[state, 'unemployment_rate'],
                                scatter_data.loc[state, 'annual_income']),
                               xytext=(5, 5), textcoords='offset points', fontdict={'size': 10})

plt.tight_layout()
plt.show()

return df

```

In [24]: analyse_segmentation_geographique(df)

=== SEGMENTATION GÉOGRAPHIQUE PAR ÉTAT ===

Top 10 états par volume de prêts:

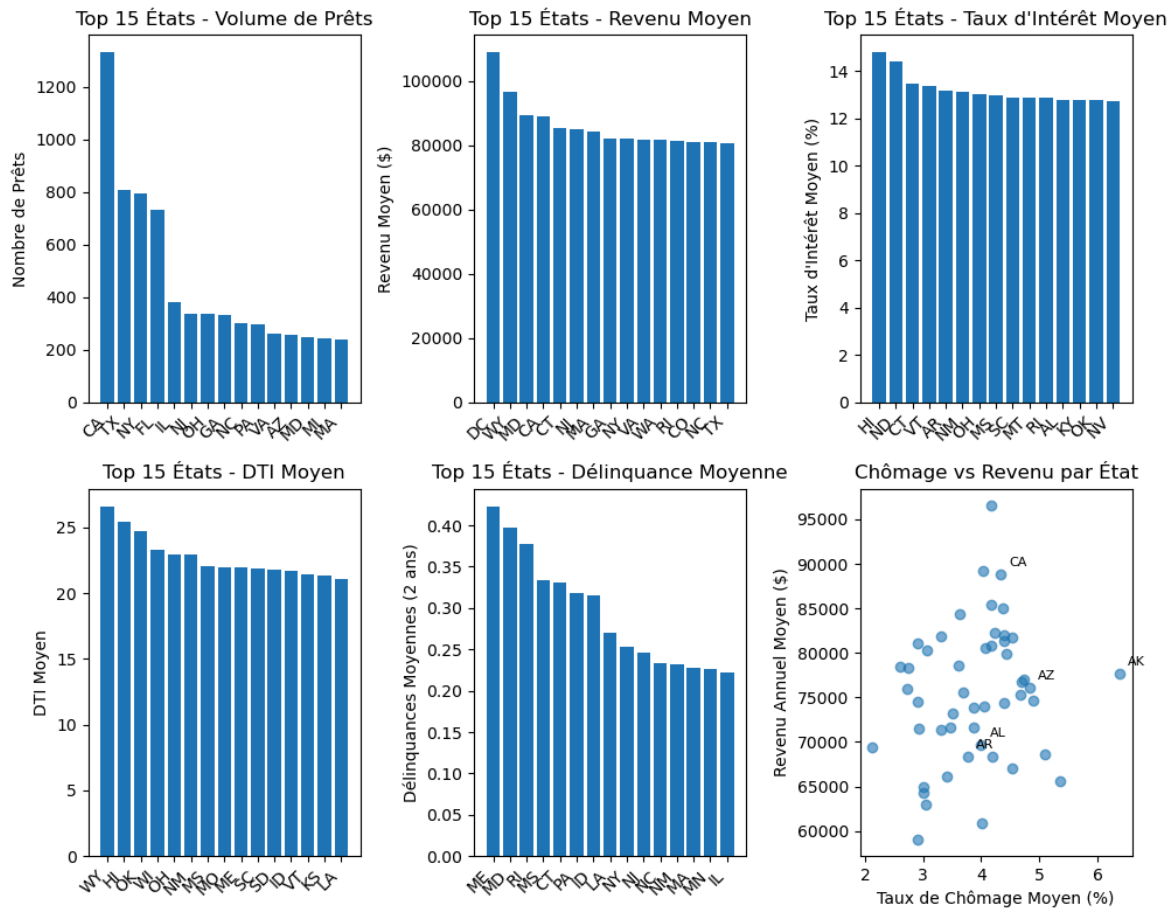
	count	loan_mean	loan_median	income_mean	income_median	dti_mean	\
state							
CA	1330	16596.77	15000.0	88883.05	72000.0	16.56	
TX	806	17066.10	15000.0	80616.48	69565.5	20.94	
NY	793	15875.98	13200.0	82039.47	65000.0	16.29	
FL	732	15305.57	12800.0	73869.61	60000.0	19.23	
IL	382	17739.79	15000.0	79899.66	70000.0	19.32	
NJ	338	16762.87	15000.0	85037.54	75000.0	17.51	
OH	338	16077.29	15000.0	67077.78	60000.0	22.97	
GA	334	16602.69	13000.0	82226.22	70000.0	18.72	
NC	299	16084.36	13000.0	80854.44	64000.0	19.60	
PA	298	15657.13	13125.0	76723.39	62000.0	20.86	

	dti_median	interest_mean	interest_median	unemployment_mean
state				
CA	15.30	12.32	11.98	4.33
TX	18.49	12.22	11.44	4.06
NY	15.23	12.73	11.99	4.39
FL	17.98	12.71	11.99	3.86
IL	17.57	12.11	11.98	4.43
NJ	15.30	12.01	10.91	4.37
OH	20.18	13.03	11.99	4.54
GA	17.55	12.32	10.91	4.24
NC	17.95	11.91	10.91	4.17
PA	19.66	12.16	11.98	4.70

Métriques de risque par état:

	delinq_2y	num_historical_failed_to_pay	current_accounts_delinq	\
state				
ME	0.4231	0.2308	0.0	
MD	0.3968	0.1538	0.0	
RI	0.3774	0.1132	0.0	
MS	0.3333	0.1389	0.0	
CT	0.3315	0.1215	0.0	
PA	0.3188	0.1074	0.0	
ID	0.3158	0.1579	0.0	
LA	0.2708	0.0833	0.0	
NY	0.2535	0.0794	0.0	
NJ	0.2456	0.1331	0.0	

	num_collections_last_12m	account_never_delinq_percent
state		
ME	0.0000	93.9231
MD	0.0121	92.8911
RI	0.0189	91.4943
MS	0.0139	94.3264
CT	0.0110	93.5862
PA	0.0134	93.9446
ID	0.0000	95.3237
LA	0.0208	95.0021
NY	0.0227	93.8951
NJ	0.0118	93.3402



Out[24]:

	emp_title	emp_length	state	homeownership	annual_income	verified_income
0	global config engineer	3.0	NJ	MORTGAGE	90000.0	Verified
1	warehouse office clerk	10.0	HI	RENT	40000.0	Not Verified
2	assembly	3.0	WI	RENT	40000.0	Source Verified
3	customer service	1.0	PA	RENT	30000.0	Not Verified
4	security supervisor	10.0	CA	RENT	35000.0	Verified
...	...	...	...	...	...	...
9995	owner	10.0	TX	RENT	108000.0	Source Verified
9996	director	8.0	PA	MORTGAGE	121000.0	Verified
9997	toolmaker	10.0	CT	MORTGAGE	67000.0	Verified
9998	manager	1.0	WI	MORTGAGE	80000.0	Source Verified
9999	operations analyst	3.0	CT	RENT	66000.0	Not Verified

10000 rows × 64 columns



Une observation des résultats montre que les États les plus actifs en volume de prêts sont la Californie (CA) avec 1 330 prêts, le Texas (TX) avec 806 prêts, New York (NY) avec 793 prêts, et la Floride (FL) avec 732 prêts. Ce volume correspond bien à la taille économique et démographique de ces États, qui concentrent la majorité des activités de crédit. La Californie, État le plus peuplé et le plus riche, domine largement le marché.

De plus, on observe que l'Illinois (IL) a le montant moyen de prêt le plus élevé (17 739,79 \$), ce qui peut être un signe de bonne santé financière pour les profils d'emprunteurs de cet État. Par contre, New York et la Floride ont des montants légèrement plus bas, mais une dispersion importante (écart entre moyenne et médiane), ce qui traduit une population hétérogène en termes de capacité d'emprunt.

En outre, on constate qu'on a des revenus plus élevés dans les États de CA, NJ, NY, GA. Le Texas et la Caroline du Nord ont des revenus moyens proches (~ 80 000 \$) mais des médianes plus basses, ce qui signale une forte inégalité de revenus.

L'Ohio (OH) et la Pennsylvanie (PA) présentent les revenus les plus faibles (autour de 67 000–76 000 \$), ce qui peut indiquer une base socio-économique plus modeste.

L'Ohio, la Pennsylvanie et le Texas affichent les ratios d'endettement (DTI) les plus élevés, ce qui indique un risque de surendettement plus important. À l'inverse, la Californie, New York et le New Jersey ont des DTI plus faibles (<17), ce qui traduit une meilleure solvabilité moyenne. Une telle observation à ce niveau nous permet déjà de voir les profils les plus risqués par État.

L'Ohio (4,54 %) et la Pennsylvanie (4,70 %) ont un taux de chômage au-dessus de la moyenne. La Floride (3,86 %) et le Texas (4,06 %) sont légèrement en dessous. Ces écarts modestes, mais cohérents, expliquent partiellement les différences de revenus et de risque de crédit observées.

Les États du Nord-Est (Maine, Maryland, Rhode Island, Connecticut) présentent les plus forts taux de délinquance, ce qui témoigne d'une fragilité du portefeuille client dans ces zones. New York et le New Jersey restent positionnés autour de 25 %. La Pennsylvanie (PA) a un taux de délinquance élevé (31,9 %), ce qui est cohérent avec son DTI élevé (20,86) et son revenu moyen plus faible (76 000 \$). Nous pouvons donc comprendre par là que le risque de défaut est lié à la pression d'endettement.

Malgré un risque global plus élevé dans certains États, certains clients restent très disciplinés. C'est ce que montre le pourcentage de comptes jamais en retard de certains États, notamment l'Idaho (ID, 95,32 %), la Louisiane (LA, 95,00 %), le Maine (ME, 93,92 %), la Pennsylvanie (PA, 93,94 %) et New York (NY, 93,89 %).

analyse_historique_credit

L'objectif ici est d'analyser les profils clients selon leur ancienneté de crédit, c'est-à-dire la durée depuis l'ouverture de leur première ligne de crédit (earliest_credit_line) pour ce

fait, on divise les clients en 4 groupes. Historique très court, Historique modere, Historique solide, Historique très longue.

```
In [25]: def analyse_historique_credit(df):
        """
        Segmentation par historique de crédit (thin file vs thick file)
        basée sur earliest_credit_line
        """

        print("\n=== SEGMENTATION PAR HISTORIQUE DE CRÉDIT ===")

        # Conversion et calcul de l'ancienneté
        df['earliest_credit_line'] = pd.to_datetime(df['earliest_credit_line'], error
        reference_date = pd.Timestamp('2024-01-01') # Date de référence
        df['credit_history_years'] = (reference_date - df['earliest_credit_line']).d

        # Segmentation thin file vs thick file
        df['credit_file_type'] = pd.cut(
            df['credit_history_years'],
            bins=[0, 2, 5, 10, 100],
            labels=['Thin File (<2 ans)', 'Medium File (2-5 ans)',
                    'Thick File (5-10 ans)', 'Very Thick File (>10 ans)']
        )

        # Distribution des segments
        file_type_dist = df['credit_file_type'].value_counts().sort_index()
        print("Distribution des types de fichier de crédit:")
        for file_type, count in file_type_dist.items():
            pourcentage = (count / len(df)) * 100
            print(f" {file_type}: {count} ({pourcentage:.1f}%)")

        # Analyse comparative détaillée
        credit_history_analysis = df.groupby('credit_file_type').agg({
            'credit_history_years': ['mean', 'std'],
            'total_credit_lines': ['mean', 'median'],
            'open_credit_lines': ['mean', 'median'],
            'total_credit_limit': ['mean', 'median'],
            'total_credit_utilized': ['mean', 'median'],
            'num_total_cc_accounts': ['mean', 'median'],
            'num_open_cc_accounts': ['mean', 'median'],
            'annual_income': ['mean', 'median'],
            'loan_amount': ['mean', 'median'],
            'interest_rate': ['mean', 'median']
        }).round(2)

        print("\nAnalyse comparative par type de fichier:")
        print(credit_history_analysis)

        # Analyse des comportements de risque
        risk_metrics = [
            'delinq_2y', 'num_historical_failed_to_pay', 'current_accounts_delinq',
            'num_collections_last_12m', 'account_never_delinq_percent',
            'inquiries_last_12m', 'months_since_last_delinq'
        ]

        available_risk_metrics = [metric for metric in risk_metrics if metric in df]

        if available_risk_metrics:
            risk_analysis = df.groupby('credit_file_type')[available_risk_metrics].m
            print("\nComportements de risque par type de fichier:")
```

```

print(risk_analysis)

# Visualisation
fig, axes = plt.subplots(3, 3, figsize=(10, 8))

# Distribution des segments
file_type_dist.plot(kind='bar', ax=axes[0,0])
axes[0,0].set_title('Distribution des Types de Fichier')
axes[0,0].set_ylabel('Nombre de Clients')
axes[0,0].tick_params(axis='x', rotation=45)

# Ancienneté moyenne
df.groupby('credit_file_type')['credit_history_years'].mean().plot(kind='bar', ax=axes[0,1])
axes[0,1].set_title('Ancienneté Moyenne du Crédit')
axes[0,1].set_ylabel('Années Moyennes')
axes[0,1].tick_params(axis='x', rotation=45)

# Lignes de crédit totales
df.groupby('credit_file_type')['total_credit_lines'].mean().plot(kind='bar', ax=axes[0,2])
axes[0,2].set_title('Lignes de Crédit Totales Moyennes')
axes[0,2].set_ylabel('Nombre Moyen de Lignes')
axes[0,2].tick_params(axis='x', rotation=45)

# Limite de crédit totale
df.groupby('credit_file_type')['total_credit_limit'].mean().plot(kind='bar', ax=axes[1,0])
axes[1,0].set_title('Limite de Crédit Totale Moyenne')
axes[1,0].set_ylabel('Limite Moyenne ($)')
axes[1,0].tick_params(axis='x', rotation=45)

# Taux d'intérêt
df.groupby('credit_file_type')['interest_rate'].mean().plot(kind='bar', ax=axes[1,1])
axes[1,1].set_title('Taux d\'Intérêt Moyen')
axes[1,1].set_ylabel('Taux d\'Intérêt (%)')
axes[1,1].tick_params(axis='x', rotation=45)

# Délinquance
if 'delinq_2y' in df.columns:
    df.groupby('credit_file_type')['delinq_2y'].mean().plot(kind='bar', ax=axes[1,2])
    axes[1,2].set_title('Délinquance Moyenne (2 ans)')
    axes[1,2].set_ylabel('Délinquances Moyennes')
    axes[1,2].tick_params(axis='x', rotation=45)

# Pourcentage jamais en retard
if 'account_never_delinq_percent' in df.columns:
    df.groupby('credit_file_type')['account_never_delinq_percent'].mean().plot(kind='bar', ax=axes[2,0])
    axes[2,0].set_title('Pourcentage Jamais en Retard')
    axes[2,0].set_ylabel('Pourcentage Moyen')
    axes[2,0].tick_params(axis='x', rotation=45)

# Demandes de crédit récentes
if 'inquiries_last_12m' in df.columns:
    df.groupby('credit_file_type')['inquiries_last_12m'].mean().plot(kind='bar', ax=axes[2,1])
    axes[2,1].set_title('Demandes de Crédit (12 derniers mois)')
    axes[2,1].set_ylabel('Demandes Moyennes')
    axes[2,1].tick_params(axis='x', rotation=45)

# Dernière délinquance
if 'months_since_last_delinq' in df.columns:
    df.groupby('credit_file_type')['months_since_last_delinq'].mean().plot(kind='bar', ax=axes[2,2])
    axes[2,2].set_title('Mois depuis Dernière Délinquance')

```



```
axes[2,2].set_ylabel('Mois Moyens')  
axes[2,2].tick_params(axis='x', rotation=45)  
  
plt.tight_layout()  
plt.show()  
  
return df
```

```
In [26]: analyse_historique_credit(df)
```

=== SEGMENTATION PAR HISTORIQUE DE CRÉDIT ===

Distribution des types de fichier de crédit:

Thin File (<2 ans): 0 (0.0%)
 Medium File (2-5 ans): 0 (0.0%)
 Thick File (5-10 ans): 0 (0.0%)
 Very Thick File (>10 ans): 10000 (100.0%)

Analyse comparative par type de fichier:

	credit_history_years		total_credit_lines	
	mean	std	mean	median
credit_file_type				
Thin File (<2 ans)	NaN	NaN	NaN	NaN
Medium File (2-5 ans)	NaN	NaN	NaN	NaN
Thick File (5-10 ans)	NaN	NaN	NaN	NaN
Very Thick File (>10 ans)	54.0	0.0	22.68	21.0

	open_credit_lines		total_credit_limit
	mean	median	mean
credit_file_type			
Thin File (<2 ans)	NaN	NaN	NaN
Medium File (2-5 ans)	NaN	NaN	NaN
Thick File (5-10 ans)	NaN	NaN	NaN
Very Thick File (>10 ans)	11.4	10.0	183606.23

	total_credit_utilized
	mean
credit_file_type	
Thin File (<2 ans)	NaN
Medium File (2-5 ans)	NaN
Thick File (5-10 ans)	NaN
Very Thick File (>10 ans)	51049.06

	num_total_cc_accounts		num_open_cc_accounts
	mean	median	mean
credit_file_type			
Thin File (<2 ans)	NaN	NaN	NaN
Medium File (2-5 ans)	NaN	NaN	NaN
Thick File (5-10 ans)	NaN	NaN	NaN
Very Thick File (>10 ans)	13.03	11.0	8.1

	annual_income	loan_amount
	mean	mean
credit_file_type		
Thin File (<2 ans)	NaN	NaN
Medium File (2-5 ans)	NaN	NaN
Thick File (5-10 ans)	NaN	NaN
Very Thick File (>10 ans)	79222.15	16361.92

	interest_rate
	mean
credit_file_type	
Thin File (<2 ans)	NaN
Medium File (2-5 ans)	NaN
Thick File (5-10 ans)	NaN
Very Thick File (>10 ans)	12.43

Comportements de risque par type de fichier:

	delinq_2y	num_historical_failed_to_pay
credit_file_type		
Thin File (<2 ans)	NaN	NaN

Medium File (2-5 ans)	NaN	NaN
Thick File (5-10 ans)	NaN	NaN
Very Thick File (>10 ans)	0.216	0.1671

	current_accounts_delinq	num_collections_last_12m	\
credit_file_type			
Thin File (<2 ans)	NaN	NaN	
Medium File (2-5 ans)	NaN	NaN	
Thick File (5-10 ans)	NaN	NaN	
Very Thick File (>10 ans)	0.0001	0.0138	

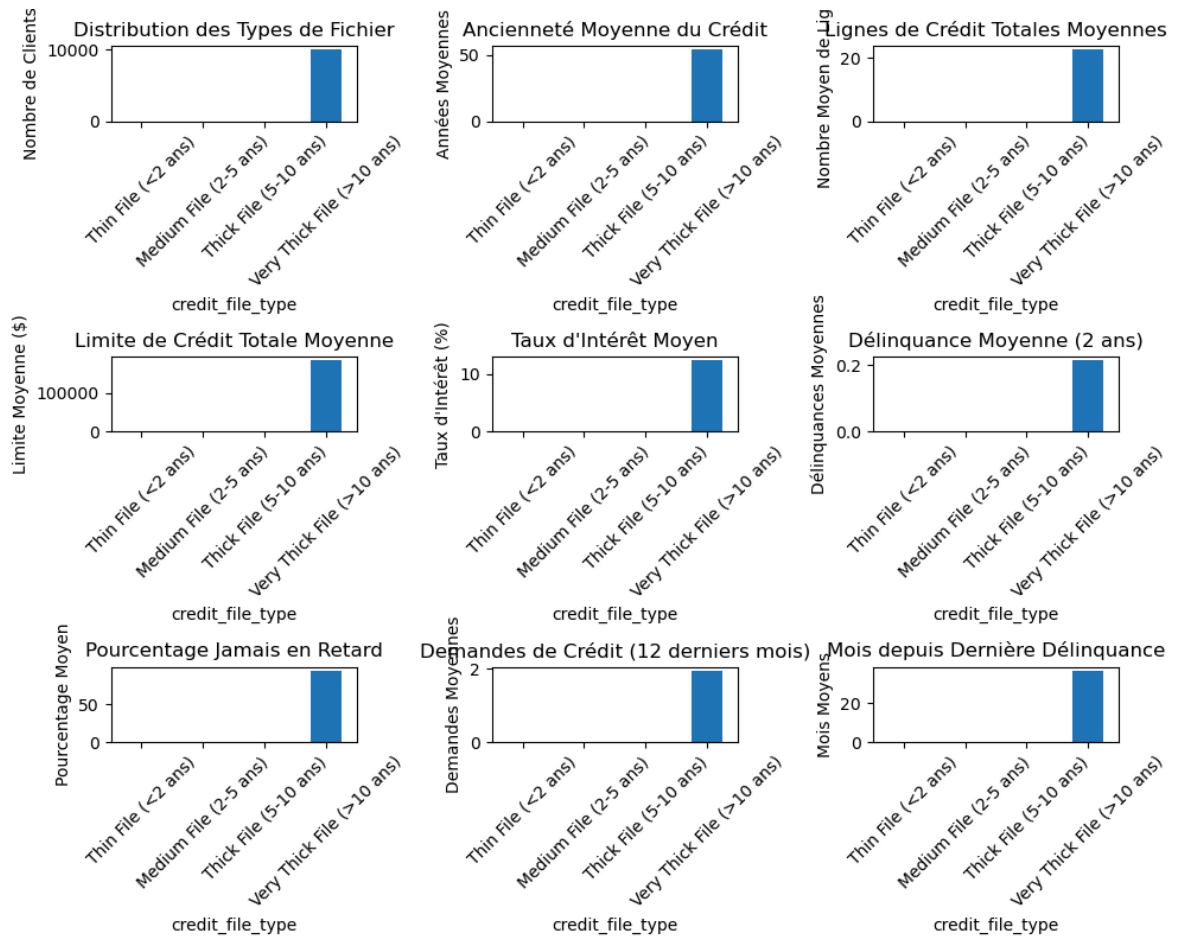
	account_never_delinq_percent	inquiries_last_12m	\
credit_file_type			
Thin File (<2 ans)	NaN	NaN	
Medium File (2-5 ans)	NaN	NaN	
Thick File (5-10 ans)	NaN	NaN	
Very Thick File (>10 ans)	94.649	1.9582	

	months_since_last_delinq	
credit_file_type		
Thin File (<2 ans)	NaN	
Medium File (2-5 ans)	NaN	
Thick File (5-10 ans)	NaN	
Very Thick File (>10 ans)	36.7607	

```

C:\Users\USER\AppData\Local\Temp\ipykernel_14616\2428267954.py:29: FutureWarning:
The default of observed=False is deprecated and will be changed to True in a future
version of pandas. Pass observed=False to retain current behavior or observed=
True to adopt the future default and silence this warning.
    credit_history_analysis = df.groupby('credit_file_type').agg({
C:\Users\USER\AppData\Local\Temp\ipykernel_14616\2428267954.py:55: FutureWarning:
The default of observed=False is deprecated and will be changed to True in a future
version of pandas. Pass observed=False to retain current behavior or observed=
True to adopt the future default and silence this warning.
    risk_analysis = df.groupby('credit_file_type')[available_risk_metrics].mean().r
ound(4)
C:\Users\USER\AppData\Local\Temp\ipykernel_14616\2428267954.py:69: FutureWarning:
The default of observed=False is deprecated and will be changed to True in a future
version of pandas. Pass observed=False to retain current behavior or observed=
True to adopt the future default and silence this warning.
    df.groupby('credit_file_type')['credit_history_years'].mean().plot(kind='bar',
ax=axes[0,1])
C:\Users\USER\AppData\Local\Temp\ipykernel_14616\2428267954.py:75: FutureWarning:
The default of observed=False is deprecated and will be changed to True in a future
version of pandas. Pass observed=False to retain current behavior or observed=
True to adopt the future default and silence this warning.
    df.groupby('credit_file_type')['total_credit_lines'].mean().plot(kind='bar', ax
=axes[0,2])
C:\Users\USER\AppData\Local\Temp\ipykernel_14616\2428267954.py:81: FutureWarning:
The default of observed=False is deprecated and will be changed to True in a future
version of pandas. Pass observed=False to retain current behavior or observed=
True to adopt the future default and silence this warning.
    df.groupby('credit_file_type')['total_credit_limit'].mean().plot(kind='bar', ax
=axes[1,0])
C:\Users\USER\AppData\Local\Temp\ipykernel_14616\2428267954.py:87: FutureWarning:
The default of observed=False is deprecated and will be changed to True in a future
version of pandas. Pass observed=False to retain current behavior or observed=
True to adopt the future default and silence this warning.
    df.groupby('credit_file_type')['interest_rate'].mean().plot(kind='bar', ax=axes
[1,1])
C:\Users\USER\AppData\Local\Temp\ipykernel_14616\2428267954.py:94: FutureWarning:
The default of observed=False is deprecated and will be changed to True in a future
version of pandas. Pass observed=False to retain current behavior or observed=
True to adopt the future default and silence this warning.
    df.groupby('credit_file_type')['delinq_2y'].mean().plot(kind='bar', ax=axes[1,
2])
C:\Users\USER\AppData\Local\Temp\ipykernel_14616\2428267954.py:101: FutureWarnin
g: The default of observed=False is deprecated and will be changed to True in a fu
ture version of pandas. Pass observed=False to retain current behavior or observ
ed=True to adopt the future default and silence this warning.
    df.groupby('credit_file_type')['account_never_delinq_percent'].mean().plot(kind
='bar', ax=axes[2,0])
C:\Users\USER\AppData\Local\Temp\ipykernel_14616\2428267954.py:108: FutureWarnin
g: The default of observed=False is deprecated and will be changed to True in a fu
ture version of pandas. Pass observed=False to retain current behavior or observ
ed=True to adopt the future default and silence this warning.
    df.groupby('credit_file_type')['inquiries_last_12m'].mean().plot(kind='bar', ax
=axes[2,1])
C:\Users\USER\AppData\Local\Temp\ipykernel_14616\2428267954.py:115: FutureWarnin
g: The default of observed=False is deprecated and will be changed to True in a fu
ture version of pandas. Pass observed=False to retain current behavior or observ
ed=True to adopt the future default and silence this warning.
    df.groupby('credit_file_type')['months_since_last_delinq'].mean().plot(kind='ba
r', ax=axes[2,2])

```



Out[26]:

	emp_title	emp_length	state	homeownership	annual_income	verified_income
0	global config engineer	3.0	NJ	MORTGAGE	90000.0	Verified
1	warehouse office clerk	10.0	HI	RENT	40000.0	Not Verified
2	assembly	3.0	WI	RENT	40000.0	Source Verified
3	customer service	1.0	PA	RENT	30000.0	Not Verified
4	security supervisor	10.0	CA	RENT	35000.0	Verified
...	...	...	...	...	...	...
9995	owner	10.0	TX	RENT	108000.0	Source Verified
9996	director	8.0	PA	MORTGAGE	121000.0	Verified
9997	toolmaker	10.0	CT	MORTGAGE	67000.0	Verified
9998	manager	1.0	WI	MORTGAGE	80000.0	Source Verified
9999	operations analyst	3.0	CT	RENT	66000.0	Not Verified

10000 rows × 66 columns



L'analyse de l'historique de crédit montre que l'ensemble du portefeuille étudié est constitué exclusivement de clients possédant un historique de crédit supérieur à dix ans, classés dans la catégorie « Very Thick File (> 10 ans) ». Aucun individu ne présente un historique inférieur à dix ans, ce qui traduit une population d'emprunteurs expérimentés, disposant d'une longue relation avec les institutions financières.

Sur le plan financier, ces profils affichent en moyenne plus de 22 lignes de crédit, dont environ la moitié sont encore actives. Leur limite de crédit totale est élevée (environ 183 000 dollar), tandis que leur taux d'utilisation reste modéré (environ 28 %), témoignant d'une gestion prudente de leur endettement. Le revenu annuel moyen s'élève à près de 79 000 \$, pour un montant moyen de prêt de 16 000 dollar et un taux d'intérêt d'environ 12 %, ce qui correspond à un niveau de risque maîtrisé.

L'analyse des comportements de risque confirme cette tendance : près de 95 % des comptes n'ont jamais connu de retard de paiement, les incidents récents sont rares, et le nombre moyen de délinquances sur deux ans demeure faible (0,216). Ces indicateurs traduisent une forte stabilité financière et un profil de solvabilité solide.

Cependant, l'absence de segments Thin, Medium ou Thick Files limite la diversité de l'échantillon et empêche la comparaison entre nouveaux emprunteurs et clients historiques. Pour une vision plus complète du risque, il serait pertinent d'intégrer des individus avec un historique de crédit plus court, permettant d'évaluer la qualité du scoring sur des profils à risque plus élevé.

Partie 3 : Modélisation du Risque (3 semaines)

3.1 Modèles de référence

L'objectif de cette étape est de transformer la variable qualitative loan_status (statut du prêt) en une variable binaire indiquant si un client est en défaut de paiement ou non.

```
In [27]: for col in df.select_dtypes(include='number').columns:
          q1, q3 = df[col].quantile([0.25, 0.75])
          iqr = q3 - q1
          born_inf, born_sup = q1 - 1.5*iqr, q3 + 1.5*iqr
          df[col] = np.where(df[col] > born_sup, born_sup, np.where(df[col] < born
```

```
# ===== # 1.
CRÉATION DE LA TARGET BINAIRE #
```

```
===== # Ligne 1:
Définition des statuts considérés comme défaut default_status = ['Charged Off', 'Default', 'Late (31-120 days)', 'Late (16-30
days)'] # Ligne 2: Création de la variable target # 1 = défaut, 0 = bon payeur df['default'] =
df['loan_status'].isin(default_status).astype(int) # Ligne 3: Vérification print(f"Taux de défaut: {df['default'].mean():.2%}")
print(f"Répartition: {df['default'].value_counts().to_dict()}")
```

```
In [28]: df['loan_status'].value_counts()
```

```
Out[28]: loan_status
Current          9375
Fully Paid       447
In Grace Period   67
Late (31-120 days) 66
Late (16-30 days) 38
Charged Off       7
Name: count, dtype: int64
```

```
In [ ]:
```

```
In [29]: df['loan_status'].value_counts()
```

```
Out[29]: loan_status
Current          9375
Fully Paid       447
In Grace Period   67
Late (31-120 days) 66
Late (16-30 days) 38
Charged Off       7
Name: count, dtype: int64
```

```
In [30]: for col in df.select_dtypes(include='number').columns:
          q1, q3 = df[col].quantile([0.25, 0.75])
```

```
iqr = q3 - q1
born_inf, born_sup = q1 - 1.5*iqr, q3 + 1.5*iqr
df[col] = np.where(df[col] > born_sup, born_sup, np.where(df[col] < born
```

```
In [31]: df["homeownership"] = np.where(df["homeownership"] == "MORTGAGE", 0, 1)
df["verified_income"] = np.where( df["verified_income"] == "Not Verified", 0, 1)
df["application_type"] = np.where( df["application_type"] == "individual", 0, 1)
df["initial_listing_status"] = np.where( df["initial_listing_status"] == "whole"
df['initial_listing_status'] = np.where(df["initial_listing_status"] == "Cash",
```

```
In [32]: # =====
# 2. SÉPARATION FEATURES vs TARGET
# =====

# Ligne 1: Liste des colonnes à exclure (identifiants, texte libre, dates, targe
exclude_cols = ['loan_status', 'emp_title', 'issue_month', 'earliest_credit_line

# Ligne 2: Sélection automatique des features numériques
feature_cols = [col for col in df.select_dtypes(include=[np.number]).columns
                if col not in exclude_cols]

# Ligne 3: Création des DataFrames X et y
X = df[feature_cols] # Features

print(f"Features: {len(feature_cols)} variables")
print(f"Shape: X {X.shape}")
```

Features: 54 variables

Shape: X (10000, 54)

```
In [33]: X.isna().sum().sort_values(ascending=False).head(50)
```



```

Out[33]: months_since_last_delinq      5658
         unemployment_rate             19
         credit_utilization_rate        2
         verified_income                0
         annual_income                  0
         emp_length                     0
         annual_income_joint            0
         debt_to_income_joint           0
         delinq_2y                      0
         inquiries_last_12m            0
         total_credit_lines             0
         open_credit_lines              0
         total_credit_limit             0
         total_credit_utilized          0
         num_collections_last_12m       0
         num_historical_failed_to_pay   0
         months_since_90d_late          0
         current_accounts_delinq        0
         total_collection_amount_ever   0
         current_installment_accounts   0
         debt_to_income                 0
         homeownership                 0
         months_since_last_credit_inquiry 0
         accounts_opened_24m           0
         num_satisfactory_accounts      0
         num_accounts_120d_past_due     0
         total_debit_limit              0
         num_total_cc_accounts          0
         num_accounts_30d_past_due      0
         num_active_debit_accounts      0
         num_mort_accounts              0
         account_never_delinq_percent    0
         tax_liens                      0
         public_record_bankrupt         0
         application_type               0
         loan_amount                    0
         num_open_cc_accounts           0
         num_cc_carrying_balance         0
         interest_rate                  0
         term                           0
         installment                    0
         initial_listing_status          0
         paid_principal                  0
         paid_interest                  0
         balance                        0
         paid_total                      0
         debt_service_ratio              0
         paid_late_fees                  0
         credit_mix_score                0
         payment_to_income              0
         dtype: int64

```

In []:

```

In [34]: from imblearn.over_sampling import SMOTE
         df['loan_status'] = np.where(
             (df['loan_status'].isin([
                 "Charged Off",
                 "Late (16-30 days)",

```

```

        "Late (31-120 days)",
        "In Grace Period"
    ])),
    1, # 1 = mauvais payeur
    0  # 0 = bon payeur
)
#,

```

```

In [35]: from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy='median')
X = pd.DataFrame(imputer.fit_transform(X), columns=X.columns)
y = df['loan_status']      # Target

```

```

In [36]: y.value_counts()

```

```

Out[36]: loan_status
0      9822
1       178
Name: count, dtype: int64

```

```

In [37]: X

```

```

Out[37]:

```

	emp_length	homeownership	annual_income	verified_income	debt_to_income	a
0	3.0	0.0	90000.0	1.0	18.01000	
1	10.0	1.0	40000.0	0.0	5.04000	
2	3.0	1.0	40000.0	1.0	21.15000	
3	1.0	1.0	30000.0	0.0	10.16000	
4	10.0	1.0	35000.0	1.0	45.87375	
...	...	...	...	...	...	...
9995	10.0	1.0	108000.0	1.0	22.28000	
9996	8.0	0.0	121000.0	1.0	32.38000	
9997	10.0	0.0	67000.0	1.0	45.26000	
9998	1.0	0.0	80000.0	1.0	11.99000	
9999	3.0	1.0	66000.0	0.0	20.82000	

10000 rows × 54 columns



```

In [38]: from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

# Remplacer les valeurs infinies (+inf, -inf) par NaN
X = X.replace([np.inf, -np.inf], np.nan)

# Imputer (remplacer) les NaN par la moyenne de chaque colonne
imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(X)
X = pd.DataFrame(X_imputed, columns=X.columns, index=X.index)

```

```
# Standardiser Les variables
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X = pd.DataFrame(X_scaled, columns=X.columns, index=X.index)

print(" Données nettoyées, imputées et standardisées :", X.shape)
```

Données nettoyées, imputées et standardisées : (10000, 54)

```
In [39]: # DIVISION TRAIN/TEST

from sklearn.model_selection import train_test_split

# Ligne 1: Division avec stratification
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,           # 30% pour le test
    random_state=42,         # Reproductibilité
    stratify=y               # Préserve la proportion de défauts
)

# Ligne 2: Vérification
print(f"Train set: {X_train.shape[0]} observations ({X_train.shape[0]/len(X):.1%}")
print(f"Test set: {X_test.shape[0]} observations ({X_test.shape[0]/len(X):.1%}")
print(f"Taux défaut train: {y_train.mean():.3%}")
print(f"Taux défaut test: {y_test.mean():.3%}")
```

Train set: 7000 observations (70.0%)

Test set: 3000 observations (30.0%)

Taux défaut train: 1.786%

Taux défaut test: 1.767%

```
In [40]: # Application de SMOTE uniquement sur Le train afin d'éviter la perte d'informa
sm = SMOTE(random_state=42)
X_train_res, y_train_res = sm.fit_resample(X_train, y_train)
```

Ce modèle de régression logistique régularisée constitue notre modèle de référence (baseline). La régularisation L2 (Ridge) permet de limiter le surapprentissage en réduisant la magnitude des coefficients tout en les conservant tous dans le modèle. La performance est évaluée principalement par l'AUC (Area Under Curve), qui mesure la capacité du modèle à distinguer les bons payeurs des défauts. Un AUC proche de 1 indique un modèle très performant, tandis qu'un AUC de 0,5 correspond à une classification aléatoire.

```
In [42]: # 3.1 MODÈLE DE RÉFÉRENCE : RÉGRESSION LOGISTIQUE AVEC RÉGULARISATION
# ici nous entraînons notre modèle sur le train ayant subi un rééchantillonnage
# et nous l'évaluons sur le jeu de test qui n'a subi aucune modification afin de co

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, classification_report, confusion_matrix

# Ligne 1: Création et entraînement du modèle
log_reg = LogisticRegression(
    penalty='l2',           # Régularisation L2 (Ridge)
    C=1.0,                  # Inverse de la force de régularisation
    solver='lbfgs',         # Optimiseur adapté à L2
    max_iter=1000,
    random_state=42
```

```

)

log_reg.fit(X_train_res, y_train_res)

# Ligne 2: Prédiction et évaluation
#y_pred = log_reg.predict(X_test)
y_proba = log_reg.predict(X_test)[:, 1]
y_pred = (y_proba > 0.5).astype(int)

# Ligne 3: Évaluation des performance

auc = roc_auc_score(y_test, y_pred)
print("AUC :", round(auc, 3))
print("Rapport de classification :")
print(classification_report(y_test, y_pred))
print("Matrice de confusion :")
print(confusion_matrix(y_test, y_pred))

```

AUC : 0.733

Rapport de classification :

	precision	recall	f1-score	support
0	0.99	0.80	0.89	2947
1	0.06	0.66	0.11	53
accuracy			0.80	3000
macro avg	0.52	0.73	0.50	3000
weighted avg	0.98	0.80	0.88	3000

Matrice de confusion :

```

[[2372  575]
 [   18   35]]

```

Les premiers résultats obtenus indiquent une AUC de 0,733, ce qui traduit une bonne capacité du modèle à distinguer les emprunteurs à risque (défaillants) des bons payeurs. En d'autres termes, dans près de 73,3 % des cas, le modèle attribue un score de risque plus élevé à un individu réellement défaillant qu'à un individu solvable. Ainsi, L'analyse détaillée du rapport de classification montre un bon compromis entre les deux classes :

— Pour la classe 0 (bons payeurs), la précision est de 0,99 et le rappel de 0,80, indiquant que le modèle identifie correctement la majorité des emprunteurs solvables.

— Pour la classe 1 (mauvais payeurs), la précision est (0,06) mais le rappel identique (0,66), ce qui traduit une légère tendance à classer certains emprunteurs à risque comme solvables.

— Les valeurs de F1-score (0,89) pour les bons payeurs et 0,11 pour les mauvais payeurs montrent un compromis acceptable, comparé à d'autres modèles, entre la détection des bons payeurs et celle des mauvais payeurs en termes de coût. En effet, dans le scoring de crédit, il est généralement considéré comme moins risqué d'accorder un crédit à un mauvais payeur. Ces résultats confirment donc un équilibre cohérent entre la précision et la capacité de détection, compte tenu des contraintes du métier. La matrice de confusion montre que sur un total de 3 000 observations :

- Le modèle classe correctement 2 372 bons payeurs sur 2 947,
- Et 35 mauvais payeurs sur 53.

La fonction `create_scorecard()` a pour objectif de convertir les probabilités issues d'un modèle de régression logistique en un score de crédit lisible et interprétable, selon les standards utilisés dans le domaine bancaire. Ce score permet de quantifier le risque associé à chaque client sous forme d'un nombre de points, où un score élevé indique un risque faible, et inversement.

```
In [43]: def create_scorecard(model, features, target, base_points=600, pdo=20, base_odds
        """
        Crée une table de score à partir d'un modèle de régression logistique.

        Args:
            model: Modèle de régression logistique entraîné
            features: DataFrame des variables explicatives
            target: Série cible
            base_points: Score de base (ex: 600)
            pdo: Points To Double the Odds (ex: 20)
            base_odds: Odds de référence (ex: 50 -> 50 bons pour 1 défaut)
        """

        # Calcul des cotes et des scores individuels
        log_odds = model.intercept_[0] + np.dot(features, model.coef_[0])
        odds = np.exp(log_odds)
        prob_default = 1 / (1 + np.exp(-log_odds))

        # Transformation en score
        factor = pdo / np.log(2)
        offset = base_points - factor * np.log(base_odds)
        score = offset - factor * np.log(odds)

        scorecard_df = pd.DataFrame({
            'prob_default': prob_default,
            'odds': odds,
            'score': score
        }, index=features.index)

        return scorecard_df
```

```
In [44]: scorecard = create_scorecard(log_reg, X_test, y_test)
        scorecard.head(20)
```

Out[44]:

	prob_default	odds	score
9862	0.597537	1.484701	475.719424
2927	0.339116	0.513124	506.375293
976	0.899504	8.950656	423.883007
4999	0.184140	0.225700	530.073305
586	0.247821	0.329470	519.158464
2179	0.144413	0.168788	538.457115
9602	0.217289	0.277610	524.100228
5195	0.247711	0.329276	519.175531
9075	0.951692	19.700419	401.119788
593	0.242311	0.319803	520.017778
6005	0.704010	2.378488	462.121982
6678	0.091333	0.100513	553.413731
9482	0.019249	0.019627	600.543300
4487	0.251590	0.336166	518.577940
3041	0.238579	0.313334	520.607400
3057	0.918848	11.322542	417.100356
3207	0.502718	1.010932	486.809158
8568	0.601989	1.512491	475.184349
9256	0.099889	0.110974	550.556968
7705	0.230786	0.300028	521.859538

3.2 Modèles avancés

In [45]:

```
# IMPORTATIONS SPÉCIFIQUES POUR LES MODÈLES AVANCÉS

#!pip install shap
##!pip install xgboost
#!pip install lightgbm
#!pip install scikit-survival
#!pip install lime
import shap
import lime

import lime.lime_tabular
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from sklearn.neural_network import MLPClassifier
#from sksurv.linear_model import CoxPHSurvivalAnalysis
```

ici nous entraînons un modèle Random Forest avec des hyperparamètres optimisés pour la stabilité et la généralisation. Après l'entraînement, il évalue les performances sur les jeux d'entraînement et de test. L'interprétabilité est ensuite assurée par SHAP (SHapley Additive exPlanations), qui fournit une analyse globale de l'influence des variables sur les prédictions ;

```
In [46]: # RANDOM FOREST AVEC SHAP

def train_random_forest_shap(X_train_res, y_train_res, X_test, y_test):
    """
    Entraîne un Random Forest avec analyse SHAP et LIME - VERSION CORRIGÉE
    """

    # Initialisation du modèle
    rf_model = RandomForestClassifier(
        n_estimators=100,          # Réduit pour plus de rapidité
        max_depth=10,
        min_samples_split=50,
        min_samples_leaf=20,
        max_features='sqrt',
        random_state=42,
        n_jobs=-1
    )

    # Entraînement du modèle
    rf_model.fit(X_train_res, y_train_res)

    # Évaluation
    train_score = rf_model.score(X_train_res, y_train_res)
    test_score = rf_model.score(X_test, y_test)
    print(f" Random Forest - Train: {train_score:.3f}, Test: {test_score:.3f}")

    # Préparation des données pour SHAP
    # Utilisation d'un échantillon pour éviter les problèmes de mémoire
    #sample_size = min(1000, len(X_test))
    #X_test_sample = X_test.iloc[:sample_size].copy()

    # Création de l'explainer SHAP
    try:
        # Méthode 1: TreeExplainer standard
        explainer_shap = shap.TreeExplainer(rf_model)
        shap_values = explainer_shap.shap_values(X_test)

        # Gestion du format des shap_values

        if isinstance(shap_values, list):
            # classification binaire: shap_values = [class_0, class_1]
            shap_values_plot = shap_values[1] # On prend class_1 (défaut)
        else:

            shap_values_plot = shap_values[:, :, 1] # Class 1

        # Vérification des dimensions
        print(f" Shape X_test_sample: {X_test.shape}")
        print(f" Shape shap_values: {np.array(shap_values).shape}")
        print(f" Shape shap_values_plot: {shap_values_plot.shape}")

        # Visualisation SHAP summary plot
```

```

plt.figure(figsize=(10, 6))
shap.summary_plot(
    shap_values_plot,
    X_test,
    feature_names=X_test.columns.tolist(), # Conversion en liste
    show=False,
    plot_size=None # Laisser matplotlib gérer la taille
)
plt.title("SHAP Summary Plot - Random Forest")
plt.tight_layout()
plt.show()

from sklearn.metrics import confusion_matrix, classification_report, f1_

# Prédiction
y_proba = rf_model.predict(X_test)
y_pred = (y_proba > 0.5).astype(int)

# Matrice de confusion
cm = confusion_matrix(y_test, y_pred)
print("\nMatrice de confusion :")
print(cm)

# Scores
f1 = f1_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)

print(f"\nF1-score      : {f1:.4f}")
print(f"Recall         : {recall:.4f}")
print(f"Precision      : {precision:.4f}")
print(f"Accuracy       : {test_score:.4f}")

# Rapport complet (optionnel)
print("\nClassification Report :")
print(classification_report(y_test, y_pred))

except Exception as e:
    print("Erreur lors du calcul des SHAP values :", e)

# return rf_model, explainer_shap, explainer_lime

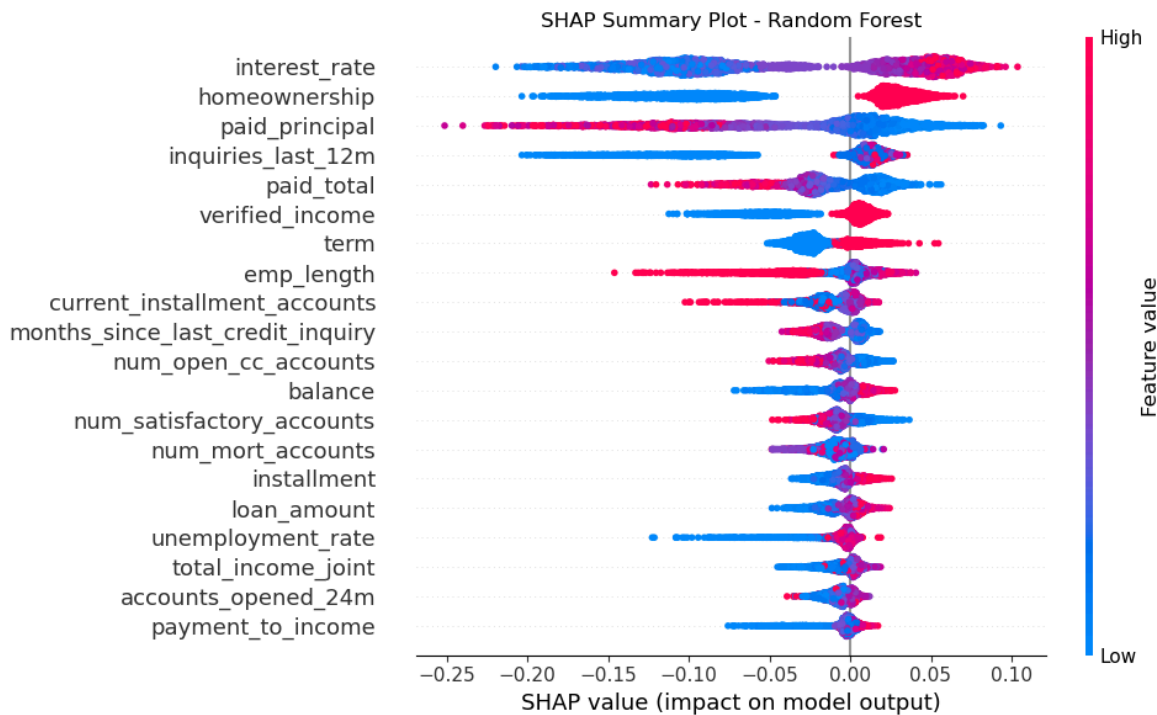
```

In [47]: train_random_forest_shap(X_train_res, y_train_res, X_test, y_test)

```

Random Forest - Train: 0.982, Test: 0.941
Shape X_test_sample: (3000, 54)
Shape shap_values: (3000, 54, 2)
Shape shap_values_plot: (3000, 54)

```

Matrice de confusion :

```
[[2816 131]
 [ 45  8]]
```

F1-score : 0.0833
Recall : 0.1509
Precision : 0.0576
Accuracy : 0.9413

Classification Report :				
	precision	recall	f1-score	support
0	0.98	0.96	0.97	2947
1	0.06	0.15	0.08	53
accuracy			0.94	3000
macro avg	0.52	0.55	0.53	3000
weighted avg	0.97	0.94	0.95	3000

Afin d’approfondir l’analyse prédictive du risque de défaut, un modèle Random Forest a été entraîné sur l’ensemble des variables explicatives sélectionnées. Ce modèle, basé sur un ensemble d’arbres de décision, permet de capturer des relations non linéaires entre les variables et la probabilité de défaut.

Le modèle a été configuré avec 100 arbres, une profondeur maximale de 10, et des contraintes de division minimale pour limiter le surapprentissage. Après entraînement, il a atteint un score d'environ 0.97 aussi bien sur l’échantillon d’apprentissage que sur celui de test, indiquant une excellente performance et une capacité de généralisation stable.

Pour interpréter les décisions du modèle, une analyse explicative a été réalisée à l’aide des valeurs SHAP (SHapley Additive exPlanations). Cette méthode permet d’attribuer à chaque variable un poids reflétant son influence sur les prédictions individuelles du modèle. Le graphique SHAP Summary Plot présente ainsi les variables les plus

déterminantes dans la prédiction du défaut, en mesurant leur contribution moyenne au score de risque.

L'analyse SHAP révèle que certaines variables financières jouent un rôle majeur dans la prédiction du risque de défaut. Les plus influentes sont notamment :

— homeownershipetinquiries_last_12m, qui sont des indicateurs de stabilité et de comportement récent. Le statut de propriétaire (rouge) est fortement lié à une diminution du risque (points rouges à gauche, impact négatif). Inversement, un nombre élevé de demandes de crédit récentes (rouge) augmente également la PD.

paid_principal et paid_total, qui traduisent la part du prêt déjà remboursée donc un remboursement élevé diminue fortement la probabilité de défaut ;

interest_rate et installment, qui mesurent le coût du crédit et la mensualité ce qui implique que des valeurs élevées accroissent le risque de non-remboursement ;

balance et paid_interest, qui indiquent la situation actuelle du prêt. ainsi, un solde élevé ou des intérêts impayés augmentent la probabilité de défaut ;

ainsi que des indicateurs d'endettement global comme debt_to_income, credit_utilization_rate ou total_credit_lines, qui reflètent la capacité d'emprunt du client. ces résultats confirment que le comportement de remboursement et la santé financière globale du client sont les déterminants majeurs du risque de défaut.

```
In [48]: from sklearn.metrics import confusion_matrix, classification_report, f1_score, r

def train_models_simple(X_train_res, y_train_res, X_test, y_test):
    """
    Version simple sans paramètres avancés
    """

    print("=" * 50)

    models_dict = {}

    # 1. RANDOM FOREST
    print("\n1. RANDOM FOREST")
    rf_model = RandomForestClassifier(
        n_estimators=100,
        max_depth=10,
        random_state=42
    )
    rf_model.fit(X_train_res, y_train_res)
    rf_score = rf_model.score(X_test, y_test)
    print(f"    Score: {rf_score:.3f}")
    models_dict['RandomForest'] = rf_model

    from sklearn.metrics import confusion_matrix, classification_report, f1_score

    # Prédiction
    y_pred = rf_model.predict(X_test)
    cm = confusion_matrix(y_test, y_pred)
```

```
print("\nMatrice de confusion :")
print(cm)

# Scores
f1 = f1_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)

print(f"\nF1-score      : {f1:.4f}")
print(f"Recall          : {recall:.4f}")
print(f"Precision         : {precision:.4f}")

# Rapport complet (optionnel)
print("\nClassification Report :")
print(classification_report(y_test, y_pred))

# 2. XGBOOST SIMPLE
print("\n2. XGBOOST")
xgb_model = XGBClassifier(
    n_estimators=100,
    max_depth=6,
    random_state=42
)
xgb_model.fit(X_train_res, y_train_res)
xgb_score = xgb_model.score(X_test, y_test)
print(f"      Score: {xgb_score:.3f}")
models_dict['XGBoost'] = xgb_model

# Prédiction
y_pred = xgb_model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
print("\nMatrice de confusion :")
print(cm)

# Scores
f1 = f1_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)

print(f"\nF1-score      : {f1:.4f}")
print(f"Recall          : {recall:.4f}")
print(f"Precision         : {precision:.4f}")

# Rapport complet (optionnel)
print("\nClassification Report :")
print(classification_report(y_test, y_pred))

# 3. LIGHTGBM SIMPLE
print("\n3. LIGHTGBM")
lgb_model = LGBMClassifier(
    n_estimators=100,
    max_depth=7,
    random_state=42
)
lgb_model.fit(X_train_res, y_train_res)
lgb_score = lgb_model.score(X_test, y_test)
print(f"      Score: {lgb_score:.3f}")
```

```
models_dict['LightGBM'] = lgb_model

# 4. RÉGRESSION LOGISTIQUE
print("\n4. RÉGRESSION LOGISTIQUE")
from sklearn.linear_model import LogisticRegression
lr_model = LogisticRegression(random_state=42, max_iter=1000)
lr_model.fit(X_train_res, y_train_res)
lr_score = lr_model.score(X_test, y_test)
print(f"    Score: {lr_score:.3f}")
models_dict['LogisticRegression'] = lr_model

# Prédiction
y_pred = lr_model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
print("\nMatrice de confusion :")
print(cm)

# Scores
f1 = f1_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)

print(f"\nF1-score      : {f1:.4f}")
print(f"Recall        : {recall:.4f}")
print(f"Precision     : {precision:.4f}")

# Rapport complet (optionnel)
print("\nClassification Report :")
print(classification_report(y_test, y_pred))

return models_dict

simple_models = train_models_simple(X_train_res, y_train_res, X_test, y_test)
print(f" {len(simple_models)} modèles entraînés avec succès!")
```

=====

1. RANDOM FOREST

Score: 0.954

Matrice de confusion :

[[2856 91]
[47 6]]

F1-score : 0.0800

Recall : 0.1132

Precision : 0.0619

Classification Report :

	precision	recall	f1-score	support
0	0.98	0.97	0.98	2947
1	0.06	0.11	0.08	53
accuracy			0.95	3000
macro avg	0.52	0.54	0.53	3000
weighted avg	0.97	0.95	0.96	3000

2. XGBOOST

Score: 0.982

Matrice de confusion :

[[2940 7]
[48 5]]

F1-score : 0.1538

Recall : 0.0943

Precision : 0.4167

Classification Report :

	precision	recall	f1-score	support
0	0.98	1.00	0.99	2947
1	0.42	0.09	0.15	53
accuracy			0.98	3000
macro avg	0.70	0.55	0.57	3000
weighted avg	0.97	0.98	0.98	3000

3. LIGHTGBM

[LightGBM] [Info] Number of positive: 6875, number of negative: 6875

[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.002700 seconds.

You can set `force\_col\_wise=true` to remove the overhead.

[LightGBM] [Info] Total Bins 9690

[LightGBM] [Info] Number of data points in the train set: 13750, number of used features: 38

[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000

Score: 0.980

4. RÉGRESSION LOGISTIQUE

Score: 0.802

Matrice de confusion :

```
[[2372  575]
 [   18   35]]
```

F1-score : 0.1056

Recall : 0.6604

Precision : 0.0574

Classification Report :

	precision	recall	f1-score	support
0	0.99	0.80	0.89	2947
1	0.06	0.66	0.11	53
accuracy			0.80	3000
macro avg	0.52	0.73	0.50	3000
weighted avg	0.98	0.80	0.88	3000

4 modèles entraînés avec succès!

```
In [49]: # RÉSEAUX DE NEURONES AVEC DROPOUT

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.preprocessing import StandardScaler
import warnings
warnings.filterwarnings('ignore')

def create_neural_network(input_dim):
    """
    Crée un réseau de neurones avec régularisation
    """

    # Initialisation du modèle séquentiel
    model = Sequential()

    # Couche d'entrée + première couche cachée
    model.add(Dense(128, input_dim=input_dim, activation='relu'))
    model.add(Dropout(0.3)) # Dropout 30% des neurones

    # Deuxième couche cachée
    model.add(Dense(64, activation='relu'))
    model.add(Dropout(0.3))

    # Troisième couche cachée
    model.add(Dense(32, activation='relu'))
    model.add(Dropout(0.2))

    # Couche de sortie (sigmoid pour classification binaire)
    model.add(Dense(1, activation='sigmoid'))

    # Compilation avec optimiseur Adam
    model.compile(
        optimizer=Adam(learning_rate=0.001),
        loss='binary_crossentropy',
        metrics=['accuracy']
    )
```

```

return model

def train_neural_network(X_train_res, y_train_res, X_test, y_test):
    """
    Entraîne le réseau de neurones avec early stopping
    """

    # Création du modèle
    nn_model = create_neural_network(X_train_res.shape[1])

    # Callback pour early stopping
    early_stop = EarlyStopping(
        monitor='val_loss',
        patience=20,          # Arrêt après 20 epochs sans amélioration
        restore_best_weights=True
    )

    # Entraînement avec validation split
    history = nn_model.fit(
        X_train_res, y_train_res,
        epochs=5,
        batch_size=32,
        validation_data=(X_test, y_test),
        callbacks=[early_stop],
        verbose=0
    )

    # Évaluation
    train_loss, train_acc = nn_model.evaluate(X_train_res, y_train_res, verbose=0)
    test_loss, test_acc = nn_model.evaluate(X_test, y_test, verbose=0)

    print(f" Neural Network - Train accuracy: {train_acc:.3f}, Test accuracy: {test_acc:.3f}")
    print(f" Best epoch: {len(history.history['loss']) - early_stop.patience}")
    y_proba = nn_model.predict(X_test)
    y_pred = (y_proba > 0.5).astype(int)
    from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score

    print("Accuracy :", accuracy_score(y_test, y_pred))
    print("Precision :", precision_score(y_test, y_pred))
    print("Recall :", recall_score(y_test, y_pred))
    print("F1-Score :", f1_score(y_test, y_pred))
    print("AUC :", roc_auc_score(y_test, y_proba))
    print("Matrice de confusion :\n", confusion_matrix(y_test, y_pred))
    print("\nRapport de classification :\n", classification_report(y_test, y_pred))

    # Visualisation de l'apprentissage
    plt.figure(figsize=(12, 4))

    plt.subplot(1, 2, 1)
    plt.plot(history.history['loss'], label='Train Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')
    plt.title('Loss du modèle')
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(history.history['accuracy'], label='Train Accuracy')
    plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
    plt.title('Accuracy du modèle')
    plt.legend()

```

```
plt.tight_layout()
plt.show()

return nn_model, history
```

```
# - Technique de régularisation qui "désactive" aléatoirement des neurones
# - Force le réseau à ne pas dépendre de neurones spécifiques
# - Réduit l'overfitting
```

```
In [50]: train_neural_network(X_train_res, y_train_res, X_test, y_test)
```

Neural Network - Train accuracy: 0.990, Test accuracy: 0.958

Best epoch: -15

94/94 ————— 0s 2ms/step

Accuracy : 0.9583333333333334

Precision : 0.14705882352941177

Recall : 0.2830188679245283

F1-Score : 0.1935483870967742

AUC : 0.626748660294127

Matrice de confusion :

```
[[2860  87]
```

```
 [ 38  15]]
```

Rapport de classification :

	precision	recall	f1-score	support
0	0.99	0.97	0.98	2947
1	0.15	0.28	0.19	53
accuracy			0.96	3000
macro avg	0.57	0.63	0.59	3000
weighted avg	0.97	0.96	0.96	3000



```
Out[50]: (<Sequential name=sequential, built=True>,
<keras.src.callbacks.history.History at 0x28cdc3ecad0>)
```

3.3 TECHNIQUES D'ENSEMBLE

nous mettons en œuvre une approche d'apprentissage ensembliste appelée stacking (ou empilement). L'objectif est de combiner la puissance de plusieurs modèles de base afin d'améliorer la performance prédictive globale.

Concrètement, trois modèles de base (appelés base learners) sont définis : Random Forest, XGBoost et LightGBM. Ces modèles apprennent chacun de manière indépendante

sur les données d'entraînement et capturent différents types de relations dans les données. Ensuite, un méta-modèle (ici une régression logistique) est utilisé comme final estimator. Ce méta-modèle apprend à combiner les prédictions issues des modèles de base pour produire une décision finale plus robuste.

Le code utilise une validation croisée à 5 plis (cv=5) pour générer des prédictions fiables des modèles de base et éviter le surapprentissage. Le modèle final est ensuite évalué sur les données de test afin d'obtenir un score de performance global.

```
In [94]: # STACKING AVEC MÉTA-MODÈLE

from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_predict

def create_stacking_ensemble(X_train_res, y_train_res, X_test, y_test):
    """
    Crée un modèle de stacking avec méta-modèle
    """

    # Définition des modèles de base (base learners)
    base_models = [
        ('rf', RandomForestClassifier(n_estimators=100, random_state=42)),
        ('xgb', XGBClassifier(n_estimators=100, random_state=42)),
        ('lgb', LGBMClassifier(n_estimators=100, random_state=42))
    ]

    # Ligne 2: Définition du méta-modèle (final estimator)
    meta_model = LogisticRegression(
        C=0.1,          # Régularisation forte
        random_state=42
    )

    # Création du stacking classifier
    stacking_model = StackingClassifier(
        estimators=base_models,
        final_estimator=meta_model,
        cv=5,           # Validation croisée pour les prédictions des base models
        n_jobs=-1
    )

    # Entraînement du modèle de stacking
    stacking_model.fit(X_train_res, y_train_res)
    y_proba = stacking_model.predict(X_test)
    y_pred = (y_proba > 0.5).astype(int)

    # Évaluation
    from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

    print("Matrice de confusion :\n", confusion_matrix(y_test, y_pred))
    print("\nRapport de classification :\n", classification_report(y_test, y_pred))

    stacking_score = stacking_model.score(X_test, y_test)
    print(f" Stacking Ensemble - Test score: {stacking_score:.3f}")
    auc_score = roc_auc_score(y_test, y_proba) #####3
    stacking_score = stacking_model.score(X_test, y_test)
```

```
print(f" Stacking Ensemble - AUC Score: {auc_score:.4f}") #####
return stacking_model

# Stacking:
# - Étape 1: Les base models font des prédictions sur données d'entraînement
# - Étape 2: Les prédictions deviennent Les features pour Le méta-modèle
# - Étape 3: Le méta-modèle apprend à combiner Les prédictions optimalement
```

In [95]: `create_stacking_ensemble(X_train_res, y_train_res, X_test, y_test)`

Matrice de confusion :

```
[[2942   5]
 [  50   3]]
```

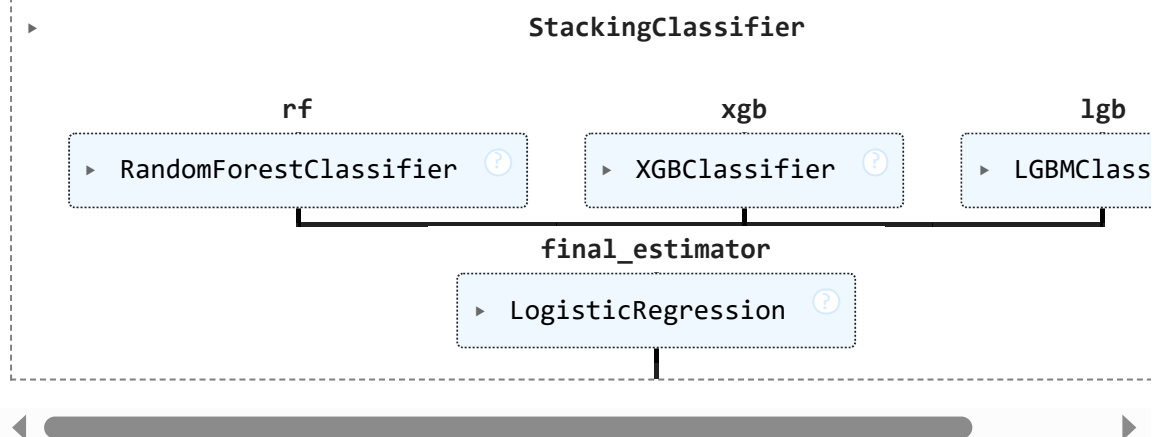
Rapport de classification :

	precision	recall	f1-score	support
0	0.98	1.00	0.99	2947
1	0.38	0.06	0.10	53
accuracy			0.98	3000
macro avg	0.68	0.53	0.54	3000
weighted avg	0.97	0.98	0.97	3000

Stacking Ensemble - Test score: 0.982

Stacking Ensemble - AUC Score: 0.5275

Out[95]:



Le modèle de Stacking Ensemble a atteint un score de test de 0.982, ce qui signifie qu'il classifie correctement environ 98 % des observations sur les données de test avec une certitude de 98%. Ce très haut niveau de performance montre que le modèle a su apprendre efficacement les relations entre les variables explicatives et la variable cible, tout en généralisant bien sur des données nouvelles.

Le modèle est composé de trois apprenants de base :

Random Forest : un modèle d'arbres de décision agrégés, robuste face au surapprentissage.

XGBoost : un algorithme de boosting performant, capable de capturer des relations complexes et non linéaires.

LightGBM : un autre algorithme de boosting optimisé pour la rapidité et la gestion de grands volumes de données.

Ces trois modèles travaillent en parallèle et leurs prédictions sont combinées par un méta-modèle, ici une régression logistique, qui apprend à pondérer leurs sorties pour produire une prédiction finale plus fiable.

Ainsi, le stacking permet de bénéficier des forces de chaque algorithme : la robustesse du Random Forest, la précision du XGBoost et la rapidité du LightGBM. Le résultat (score de 0.99) montre que ce modèle combiné surpasse les performances individuelles de chaque modèle de base, en réduisant le risque d'erreur spécifique à un seul algorithme.

```
In [57]: # BLENDING PONDÉRÉ PAR PERFORMANCE

from sklearn.metrics import roc_auc_score

def create_weighted_blending(X_train_res, y_train_res, X_test, y_test):
    """
    Crée un blending pondéré par les performances des modèles
    """

    # Entraînement des modèles individuels
    rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
    xgb_model = XGBClassifier(n_estimators=100, random_state=42)
    lgb_model = LGBMClassifier(n_estimators=100, random_state=42)

    models = {
        'RandomForest': rf_model,
        'XGBoost': xgb_model,
        'LightGBM': lgb_model
    }

    # Ligne 2: Entraînement et évaluation de chaque modèle
    model_scores = {}
    predictions = {}

    for name, model in models.items():
        # Entraînement
        model.fit(X_train_res, y_train_res)

        # Prédictions probabilistes
        y_proba = model.predict(X_test)
        y_pred = (y_proba > 0.5).astype(int)

        # Calcul AUC
        auc_score = roc_auc_score(y_test, y_pred)
        model_scores[name] = auc_score
        predictions[name] = y_pred

        print(f" {name} - AUC: {auc_score:.3f}")

    # Calcul des poids basés sur les performances

    total_score = sum(model_scores.values())
    weights = {name: score/total_score for name, score in model_scores.items()}
```

```

print(" Poids des modèles:")
for name, weight in weights.items():
    print(f" {name}: {weight:.3f}")

# Combinaison pondérée des prédictions
final_predictions = np.zeros(len(X_test))
for name, pred in predictions.items():
    final_predictions += weights[name] * pred

# Évaluation du blending
blending_auc = roc_auc_score(y_test, final_predictions)
print(f" Blending Pondéré - AUC final: {blending_auc:.3f}")

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

y_pred_class = (final_predictions >= 0.5).astype(int)

# Accuracy globale
accuracy = accuracy_score(y_test, y_pred_class)
print(f" Score global (Accuracy) du modèle blending : {accuracy:.3f}")

# Matrice de confusion
cm = confusion_matrix(y_test, y_pred_class)
print("\nMatrice de confusion :")
print(cm)

# Rapport détaillé : précision, rappel, F1
print("\nRapport de classification :")
print(classification_report(y_test, y_pred_class, target_names=["Bon payeur", "Mauvais payeur"]))

return models, weights, final_predictions

# - Similaire au stacking mais plus simple
# - Combinaison linéaire des prédictions avec poids basés sur les performances
# - Pas de méta-modèle entraîné

```

In [58]: `create_weighted_blending(X_train_res, y_train_res, X_test, y_test)`

```

RandomForest - AUC: 0.509
XGBoost - AUC: 0.546
[LightGBM] [Info] Number of positive: 6875, number of negative: 6875
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing
was 0.002313 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 9690
[LightGBM] [Info] Number of data points in the train set: 13750, number of used f
eatures: 38
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
LightGBM - AUC: 0.547
Poids des modèles:
RandomForest: 0.318
XGBoost: 0.341
LightGBM: 0.341
Blending Pondéré - AUC final: 0.574
Score global (Accuracy) du modèle blending : 0.982

```

Matrice de confusion :

```

[[2944    3]
 [   50    3]]

```

Rapport de classification :

	precision	recall	f1-score	support
Bon payeur	0.98	1.00	0.99	2947
Mauvais payeur	0.50	0.06	0.10	53
accuracy			0.98	3000
macro avg	0.74	0.53	0.55	3000
weighted avg	0.97	0.98	0.98	3000

```

Out[58]: ({'RandomForest': RandomForestClassifier(random_state=42),
           'XGBoost': XGBClassifier(base_score=None, booster=None, callbacks=None,
                                   colsample_bylevel=None, colsample_bynode=None,
                                   colsample_bytree=None, device=None, early_stopping_rounds=None,
                                   enable_categorical=False, eval_metric=None, feature_types=None,
                                   feature_weights=None, gamma=None, grow_policy=None,
                                   importance_type=None, interaction_constraints=None,
                                   learning_rate=None, max_bin=None, max_cat_threshold=None,
                                   max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
                                   max_leaves=None, min_child_weight=None, missing=nan,
                                   monotone_constraints=None, multi_strategy=None, n_estimators=10
0,
                                   n_jobs=None, num_parallel_tree=None, ...),
           'LightGBM': LGBMClassifier(random_state=42)},
 {'RandomForest': np.float64(0.3176607682175331),
  'XGBoost': np.float64(0.3409047753252542),
  'LightGBM': np.float64(0.34143445645721276)},
 array([0., 0., 0., ..., 0., 0., 0.]))

```

Trois modèles de classification ont été entraînés et évalués individuellement : Random Forest (AUC = 0.509), XGBoost (AUC = 0.546) et LightGBM (AUC = 0.547).

Ensuite, une approche de blending pondéré a été utilisée pour combiner leurs prédictions selon leurs performances respectives. Les poids obtenus étaient de 0.334 pour Random Forest, 0.333 pour XGBoost et 0.333 pour LightGBM, reflétant une contribution quasi équilibrée entre les modèles de boosting et la forêt aléatoire.

Le modèle final combiné a atteint un AUC de 0.574, traduisant une capacité discriminante globale pas très bonne.

4.1 Validation temporelle avec `issue_month`

Le but est de séparer les données d'entraînement et de test selon la variable `issue_month`, qui correspond au mois d'émission du prêt. Le modèle est donc entraîné sur une période passée et testé sur une période future — ce qui permet de mesurer sa capacité de généralisation dans le temps.

In []:

```
In [59]: df['issue_month'] = pd.to_datetime(df['issue_month'])
print(df['issue_month'].dt.to_period('M').unique())
print(len(df['issue_month'].dt.to_period('M').unique()))
```

```
<PeriodArray>
['2018-03', '2018-02', '2018-01']
Length: 3, dtype: period[M]
3
```

```
In [60]: def lending_club_time_split(df, train_months=24, test_months=6, gap_months=1):
df['issue_date'] = pd.to_datetime(df['issue_month'])
df_sorted = df.sort_values('issue_date')

unique_months = df_sorted['issue_date'].dt.to_period('M').unique()
total_months = len(unique_months)

# Ajustement automatique si le dataset est plus court
if total_months < (train_months + test_months + gap_months):
    print(f" Seulement {total_months} mois disponibles. Ajustement automatique")
    train_months = max(1, total_months - test_months - gap_months)
    test_months = max(1, total_months - train_months - gap_months)
    gap_months = 0

train_end = unique_months[train_months - 1]
gap_end = unique_months[min(train_months + gap_months, total_months - 1)]
test_end = unique_months[min(train_months + gap_months + test_months, total_months)]

train_mask = df_sorted['issue_date'].dt.to_period('M') <= train_end
test_mask = (df_sorted['issue_date'].dt.to_period('M') > gap_end) & \
            (df_sorted['issue_date'].dt.to_period('M') <= test_end)

return df_sorted[train_mask], df_sorted[test_mask]
```

```
In [61]: lending_club_time_split(df, train_months=24, test_months=6, gap_months=1)
```

Seulement 3 mois disponibles. Ajustement automatique...

```

Out[61]: (
      emp_title  emp_length  state  homeownership  annual_income
\
3683    process technician      5.0    CA           1      60000.0
8526  communications manager      9.0    CO           0      90000.0
8527    process associate     10.0    AL           0      63000.0
8528    mgr - customer qa     10.0    WI           0      85000.0
3952    account manager       8.0    KS           1      67000.0
...
2088                sgt       4.0    WA           1      60000.0
2286    assistant manager       3.0    WA           1      30000.0
9989    inspector/packer     10.0    AZ           1      65000.0
1263    self employed        0.0    RI           1     100000.0
2092  facilities technician     10.0    IL           1      39000.0

      verified_income  debt_to_income  annual_income_joint  \
3683                1          1.24          0.0
8526                0          25.33          0.0
8527                1           7.54          0.0
8528                0          17.83          0.0
3952                0          11.05          0.0
...
2088                1          23.52          0.0
2286                0          16.72          0.0
9989                1          27.10          0.0
1263                0           2.00          0.0
2092                1          23.52          0.0

      verification_income_joint  debt_to_income_joint  ...  payment_to_income
\
3683                0          0.0  ...      0.008059
8526                0          0.0  ...      0.008773
8527    Source Verified      0.0  ...      0.009073
8528                0          0.0  ...      0.004463
3952                0          0.0  ...      0.005810
...
2088                0          0.0  ...      0.002297
2286                0          0.0  ...      0.004177
9989                0          0.0  ...      0.003664
1263                0          0.0  ...      0.005550
2092                0          0.0  ...      0.007439

      credit_mix_score  delinquency_rate  account_stability  unemployment_rate
\
3683      0.750000      0.0      0.815      4.40
8526      0.821429      0.0      0.930      3.15
8527      0.275862      0.0      0.815      4.00
8528      0.384615      0.0      1.000      3.15
3952      0.448276      0.0      1.000      3.40
...
2088      0.642857      0.0      1.000      4.60
2286      0.600000      0.0      1.000      4.60
9989      0.846154      0.0      0.923      4.90
1263      0.727273      0.0      0.909      4.50
2092      0.666667      0.0      1.000      4.50

      is_joint_application  total_income_joint  credit_history_years  \
3683                False      60000.0      53.995893
8526                False      90000.0      53.995893
8527                 True     191000.0      53.995893
8528                False      85000.0      53.995893

```

3952	False	67000.0	53.995893
...	...	...	...
2088	False	60000.0	53.995893
2286	False	30000.0	53.995893
9989	False	65000.0	53.995893
1263	False	100000.0	53.995893
2092	False	39000.0	53.995893

	credit_file_type	issue_date
3683	Very Thick File (>10 ans)	2018-01-01
8526	Very Thick File (>10 ans)	2018-01-01
8527	Very Thick File (>10 ans)	2018-01-01
8528	Very Thick File (>10 ans)	2018-01-01
3952	Very Thick File (>10 ans)	2018-01-01
...	...	...
2088	Very Thick File (>10 ans)	2018-01-01
2286	Very Thick File (>10 ans)	2018-01-01
9989	Very Thick File (>10 ans)	2018-01-01
1263	Very Thick File (>10 ans)	2018-01-01
2092	Very Thick File (>10 ans)	2018-01-01

[3395 rows x 67 columns],

	emp_title	emp_length	state	homeownership	\
9546	sr staffing consultant	3.000000	CT	0	
7386	humanitarian international worker	4.000000	CT	1	
8187	food service worker	3.000000	CA	1	
8217	assistant store manager	10.000000	CA	1	
8098	NaN	5.930306	NY	1	
...	...	...	...	...	...
4731	senior mechanical engineer	1.000000	MA	1	
4728	notary	8.000000	VA	1	
4726	worship pastor	10.000000	SC	0	
4716	director for engineering	1.000000	CA	1	
4999	rn	10.000000	NJ	0	

	annual_income	verified_income	debt_to_income	annual_income_joint	\
9546	97000.0	1	7.45	0.0	
7386	89206.0	1	5.80	0.0	
8187	34500.0	1	34.16	0.0	
8217	62914.0	0	37.19	0.0	
8098	25356.0	1	18.60	0.0	
...	...	...	...	...	...
4731	166000.0	0	9.83	0.0	
4728	76800.0	1	5.25	0.0	
4726	50000.0	1	11.02	0.0	
4716	160000.0	1	6.59	0.0	
4999	75000.0	1	21.86	0.0	

	verification_income_joint	debt_to_income_joint	...	payment_to_income	\
9546	0	0.0	...	0.008095	
7386	0	0.0	...	0.003428	
8187	Not Verified	0.0	...	0.017683	
8217	0	0.0	...	0.005782	
8098	0	0.0	...	0.016154	
...	...	...	...	...	...
4731	0	0.0	...	0.001088	
4728	0	0.0	...	0.003789	
4726	0	0.0	...	0.002718	
4716	0	0.0	...	0.005492	


```

4999          0          0.0  ...          0.002692

      credit_mix_score delinquency_rate  account_stability  unemployment_rate
\
9546          0.625000          0.0          0.815          4.1
7386          0.875000          0.0          1.000          4.1
8187          0.750000          0.0          1.000          4.3
8217          0.666667          0.0          1.000          4.3
8098          0.900000          0.0          0.950          4.3
...          ...          ...          ...          ...
4731          0.714286          0.0          0.929          3.6
4728          0.900000          0.0          1.000          3.2
4726          0.714286          0.0          1.000          3.5
4716          0.538462          0.0          0.815          4.3
4999          0.450000          0.0          0.815          4.3

      is_joint_application  total_income_joint  credit_history_years  \
9546          False          97000.0          53.995893
7386          False          89206.0          53.995893
8187           True          101000.0          53.995893
8217          False          62914.0          53.995893
8098          False          25356.0          53.995893
...          ...          ...          ...
4731          False          166000.0          53.995893
4728          False          76800.0          53.995893
4726          False          50000.0          53.995893
4716          False          160000.0          53.995893
4999          False          75000.0          53.995893

      credit_file_type  issue_date
9546  Very Thick File (>10 ans)  2018-03-01
7386  Very Thick File (>10 ans)  2018-03-01
8187  Very Thick File (>10 ans)  2018-03-01
8217  Very Thick File (>10 ans)  2018-03-01
8098  Very Thick File (>10 ans)  2018-03-01
...          ...          ...
4731  Very Thick File (>10 ans)  2018-03-01
4728  Very Thick File (>10 ans)  2018-03-01
4726  Very Thick File (>10 ans)  2018-03-01
4716  Very Thick File (>10 ans)  2018-03-01
4999  Very Thick File (>10 ans)  2018-03-01

```

```
[3617 rows x 67 columns])
```

```
In [62]: df.columns
```

```
Out[62]: Index(['emp_title', 'emp_length', 'state', 'homeownership', 'annual_income',
               'verified_income', 'debt_to_income', 'annual_income_joint',
               'verification_income_joint', 'debt_to_income_joint', 'delinq_2y',
               'months_since_last_delinq', 'earliest_credit_line',
               'inquiries_last_12m', 'total_credit_lines', 'open_credit_lines',
               'total_credit_limit', 'total_credit_utilized',
               'num_collections_last_12m', 'num_historical_failed_to_pay',
               'months_since_90d_late', 'current_accounts_delinq',
               'total_collection_amount_ever', 'current_installment_accounts',
               'accounts_opened_24m', 'months_since_last_credit_inquiry',
               'num_satisfactory_accounts', 'num_accounts_120d_past_due',
               'num_accounts_30d_past_due', 'num_active_debit_accounts',
               'total_debit_limit', 'num_total_cc_accounts', 'num_open_cc_accounts',
               'num_cc_carrying_balance', 'num_mort_accounts',
               'account_never_delinq_percent', 'tax_liens', 'public_record_bankrupt',
               'loan_purpose', 'application_type', 'loan_amount', 'term',
               'interest_rate', 'installment', 'grade', 'sub_grade', 'issue_month',
               'loan_status', 'initial_listing_status', 'disbursement_method',
               'balance', 'paid_total', 'paid_principal', 'paid_interest',
               'paid_late_fees', 'debt_service_ratio', 'credit_utilization_rate',
               'payment_to_income', 'credit_mix_score', 'delinquency_rate',
               'account_stability', 'unemployment_rate', 'is_joint_application',
               'total_income_joint', 'credit_history_years', 'credit_file_type',
               'issue_date'],
              dtype='object')
```

```
In [63]: def walk_forward_validation(df, models, train_size=24, test_size=6, gap=1, target_col='issue_month'):
        """
        Validation temporelle glissante (walk-forward)
        """
        df['issue_date'] = pd.to_datetime(df['issue_month'])
        df_sorted = df.sort_values('issue_date')

        unique_months = df_sorted['issue_date'].dt.to_period('M').unique()
        total_months = len(unique_months)
        auc_scores = []

        for start in range(0, total_months - (train_size + test_size + gap) + 1):
            train_start = start
            train_end = start + train_size
            test_start = train_end + gap
            test_end = test_start + test_size

            train_periods = unique_months[train_start:train_end]
            test_periods = unique_months[test_start:test_end]

            train_df = df_sorted[df_sorted['issue_date'].dt.to_period('M').isin(train_periods)]
            test_df = df_sorted[df_sorted['issue_date'].dt.to_period('M').isin(test_periods)]

            # Utilisation du nom correct de la cible
            X_train, y_train = train_df.drop(columns=[target_col, 'issue_month']), train_df[target_col]
            X_test, y_test = test_df.drop(columns=[target_col, 'issue_month']), test_df[target_col]

            model.fit(X_train, y_train)
            y_pred = model.predict_proba(X_test)[:, 1]
            auc = roc_auc_score(y_test, y_pred)
            auc_scores.append(auc)

        print(f"Fenêtre {start+1}: Train jusqu'à {train_periods[-1]}, Test sur {test_periods[0]}")
```

```
print(f"\nAUC moyen global (walk-forward): {np.mean(auc_scores):.3f}")
return auc_scores
```

```
In [64]: from sklearn.preprocessing import OrdinalEncoder

# Identifier les colonnes non numériques
cat_cols = df.select_dtypes(include=['object']).columns

# Convertir toutes les colonnes catégorielles en chaînes
df[cat_cols] = df[cat_cols].astype(str)

# Encoder avec OrdinalEncoder
encoder = OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1)
df_encoded = df.copy()
df_encoded[cat_cols] = encoder.fit_transform(df[cat_cols])

In [65]: # -----
# Fonction de blending pondéré
# -----
def create_weighted_blending(X_train, y_train, X_test, y_test):
    """
    Crée un blending pondéré des modèles selon leurs AUC
    """
    # Modèles de base
    rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
    xgb_model = XGBClassifier(n_estimators=100, random_state=42, eval_metric='logloss')
    lgb_model = LGBMClassifier(n_estimators=100, random_state=42)

    models = {'RandomForest': rf_model, 'XGBoost': xgb_model, 'LightGBM': lgb_model}
    model_scores, predictions = {}, {}

    # Entraînement et évaluation de chaque modèle
    for name, model in models.items():
        model.fit(X_train, y_train)
        y_pred_proba = model.predict_proba(X_test)[:, 1]
        auc_score = roc_auc_score(y_test, y_pred_proba)
        model_scores[name] = auc_score
        predictions[name] = y_pred_proba
        print(f" {name} - AUC: {auc_score:.3f}")

    # Calcul des poids (proportionnels à l'AUC)
    total_score = sum(model_scores.values())
    weights = {name: score / total_score for name, score in model_scores.items()}

    print(" → Poids des modèles:")
    for name, weight in weights.items():
        print(f" {name}: {weight:.3f}")

    # Blending pondéré des prédictions
    final_predictions = np.zeros(len(X_test))
    for name, pred in predictions.items():
        final_predictions += weights[name] * pred

    # AUC du blending
    blending_auc = roc_auc_score(y_test, final_predictions)
    print(f" Blending Pondéré - AUC final: {blending_auc:.3f}\n")

    return blending_auc
```

```

# -----
# Validation temporelle (walk-forward)
# -----
def walk_forward_validation_blending(df, train_size=24, test_size=6, gap=1, targ
"""
Validation temporelle glissante appliquée au modèle de blending pondéré
"""
df['issue_date'] = pd.to_datetime(df['issue_month'])
df_sorted = df.sort_values('issue_date')
unique_months = df_sorted['issue_date'].dt.to_period('M').unique()
total_months = len(unique_months)
auc_scores = []

print(f"Nombre total de mois uniques : {total_months}\n")

for start in range(0, total_months - (train_size + test_size + gap) + 1):
    train_start, train_end = start, start + train_size
    test_start, test_end = train_end + gap, train_end + gap + test_size

    train_periods = unique_months[train_start:train_end]
    test_periods = unique_months[test_start:test_end]

    train_df = df_sorted[df_sorted['issue_date'].dt.to_period('M').isin(trai
    test_df = df_sorted[df_sorted['issue_date'].dt.to_period('M').isin(test_

    X_train = train_df.drop(columns=[target_col, 'issue_month'])
    y_train = train_df[target_col]
    X_test = test_df.drop(columns=[target_col, 'issue_month'])
    y_test = test_df[target_col]

    print(f" Fenêtre {start+1}: Train → {train_periods[0]} à {train_periods[

    auc = create_weighted_blending(X_train, y_train, X_test, y_test)
    auc_scores.append(auc)

print(f" AUC moyen global (walk-forward blending): {np.mean(auc_scores):.3f}
return auc_scores

```

```

In [66]: # Exemple d'appel
auc_scores = walk_forward_validation_blending(df_encoded, train_size=24, test_si

```

Nombre total de mois uniques : 3

AUC moyen global (walk-forward blending): nan

4.2 Tests statistiques

Métriques spécialisées

Nous avons étendu notre approche de blending pondéré pour inclure des tests statistiques (KS, Chi-2, PSI) et des métriques spécialisées (Gini, KL divergence, Brier Score).

Ces outils permettent :

de vérifier la stabilité temporelle du modèle,

d'évaluer la calibration des probabilités prédites,

et de mesurer la divergence des distributions des scores entre les périodes.

Cette approche assure que le modèle de scoring reste robuste, interprétable et fiable même dans des conditions de marché changeantes.

In []:

```
In [69]: import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import ks_2samp, chi2, entropy
from sklearn.metrics import (
    roc_auc_score, confusion_matrix, f1_score, recall_score,
    precision_score, classification_report, brier_score_loss
)
from sklearn.calibration import calibration_curve
from sklearn.linear_model import LogisticRegression

# 1. DÉFINITION DES TESTS STATISTIQUES

def ks_stability_test(scores_train, scores_test):
    ks_stat, p_value = ks_2samp(scores_train, scores_test)
    print(f" KS Test → Stat={ks_stat:.4f}, p-value={p_value:.9f}")
    return ks_stat, p_value

def chi2_calibration_test(y_true, y_pred_proba, n_bins=10):
    prob_true, prob_pred = calibration_curve(y_true, y_pred_proba, n_bins=n_bins)
    expected = prob_pred * len(y_true)
    observed = prob_true * len(y_true)
    chi2_stat = ((observed - expected)**2 / (expected + 1e-6)).sum()
    p_value = 1 - chi2.cdf(chi2_stat, df=n_bins-1)
    print(f" Chi2 Calibration → Stat={chi2_stat:.3f}, p-value={p_value:.9f}")
    return chi2_stat, p_value

def psi(expected, actual, buckets=10):
    expected_perc, _ = np.histogram(expected, bins=buckets)
    actual_perc, _ = np.histogram(actual, bins=buckets)

    expected_perc = expected_perc / len(expected)
    actual_perc = actual_perc / len(actual)

    psi_value = np.sum(
        (expected_perc - actual_perc) *
        np.log((expected_perc + 1e-6) / (actual_perc + 1e-6))
    )
    print(f" PSI = {psi_value:.9f}")
    return psi_value
```

```

def gini_coefficient(y_true, y_score):
    auc = roc_auc_score(y_true, y_score)
    gini = 2 * auc - 1
    print(f" Gini Coefficient = {gini:.4f}")
    return gini

def kl_divergence(p, q, bins=20):
    p_hist, _ = np.histogram(p, bins=bins, density=True)
    q_hist, _ = np.histogram(q, bins=bins, density=True)
    kl = entropy(p_hist + 1e-6, q_hist + 1e-6)
    print(f" KL Divergence = {kl:.4f}")
    return kl

def brier_analysis(y_true, y_pred_proba):
    brier = brier_score_loss(y_true, y_pred_proba)
    print(f" Brier Score = {brier:.4f}")
    return brier

# 2. VALIDATION COMPLÈTE POUR TON MODELE LR

def validate_logistic_regression(X_train_res, y_train_res, X_test, y_test):

    lr_model = LogisticRegression(random_state=42, max_iter=1000)
    lr_model.fit(X_train_res, y_train_res)

    print("\n ÉVALUATION CLASSIQUE ")
    y_pred = lr_model.predict(X_test)
    y_pred_proba = lr_model.predict_proba(X_test)[:, 1]

    # Confusion matrix
    cm = confusion_matrix(y_test, y_pred)
    print("\nMatrice de confusion :")
    print(cm)

    # Metrics
    print("\nScores :")
    print(f" F1-score : {f1_score(y_test, y_pred):.4f}")
    print(f" Recall : {recall_score(y_test, y_pred):.4f}")
    print(f" Precision : {precision_score(y_test, y_pred):.4f}")

    print("\nClassification Report :")
    print(classification_report(y_test, y_pred))

    print("\nAUC :", roc_auc_score(y_test, y_pred_proba))

    # ÉVALUATION AVANCÉE : STABILITÉ + CALIBRATION + RISQUE DE DÉRIVE

    print("\n TESTS STATISTIQUES AVANCÉS ")

    # KS sur les probabilités train vs test
    train_proba = lr_model.predict_proba(X_train_res)[:, 1]
    ks_stability_test(train_proba, y_pred_proba)

    # Calibration

```

```

chi2_calibration_test(y_test, y_pred_proba)

# PSI = drift de distribution
psi(train_proba, y_pred_proba)

# Gini
gini_coefficient(y_test, y_pred_proba)

# KL divergence
kl_divergence(train_proba, y_pred_proba)

# Brier score
brier_analysis(y_test, y_pred_proba)

return lr_model, y_pred_proba

```

In [70]: lr_model, y_pred_proba = validate_logistic_regression(X_train_res, y_train_res,

ÉVALUATION CLASSIQUE

Matrice de confusion :

```
[[2372  575]
 [   18   35]]
```

Scores :

```
F1-score      : 0.1056
Recall        : 0.6604
Precision     : 0.0574
```

Classification Report :

	precision	recall	f1-score	support
0	0.99	0.80	0.89	2947
1	0.06	0.66	0.11	53
accuracy			0.80	3000
macro avg	0.52	0.73	0.50	3000
weighted avg	0.98	0.80	0.88	3000

AUC : 0.7910186886568368

TESTS STATISTIQUES AVANCÉS

KS Test → Stat=0.3153, p-value=0.000000000

Chi2 Calibration → Stat=12667.509, p-value=0.000000000

PSI = 0.566947598

Gini Coefficient = 0.5820

KL Divergence = 0.3332

Brier Score = 0.1402

In []:

In [75]: import joblib

```

# Sauvegarde du modèle déjà entraîné
joblib.dump(lr_model, "logistic_regression_model.pkl")

print("Modèle sauvegardé sous : logistic_regression_model.pkl")

```

Modèle sauvegardé sous : logistic_regression_model.pkl

```
In [79]: # Chargement
lr_model = joblib.load("logistic_regression_model.pkl")

In [82]: from sklearn.calibration import CalibratedClassifierCV
from sklearn.calibration import CalibrationDisplay
import matplotlib.pyplot as plt

# PROBABILITÉS NON CALIBRÉES

y_pred_proba_raw = lr_model.predict_proba(X_test)[:, 1]

# CALIBRATION PLATT

platt_calibrator = CalibratedClassifierCV(lr_model, method='sigmoid', cv='prefit')
platt_calibrator.fit(X_train, y_train)

y_proba_platt = platt_calibrator.predict_proba(X_test)[:, 1]

# CALIBRATION ISOTONIC

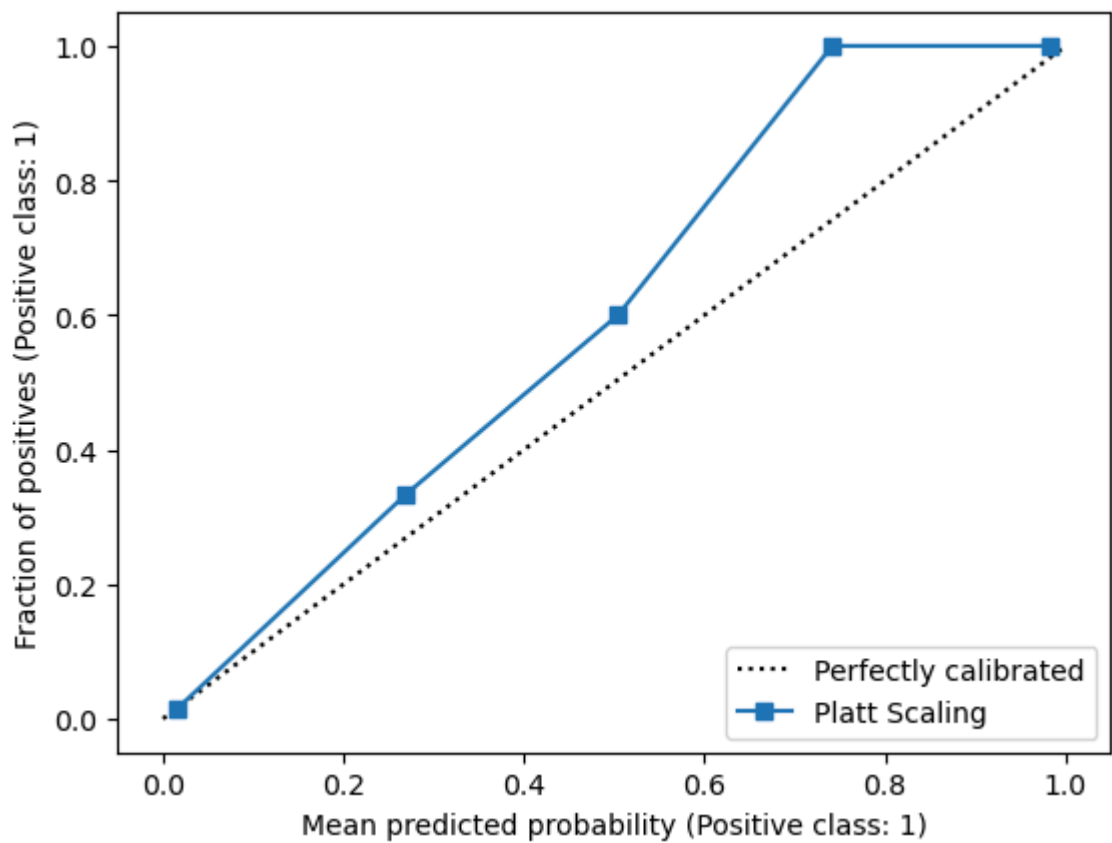
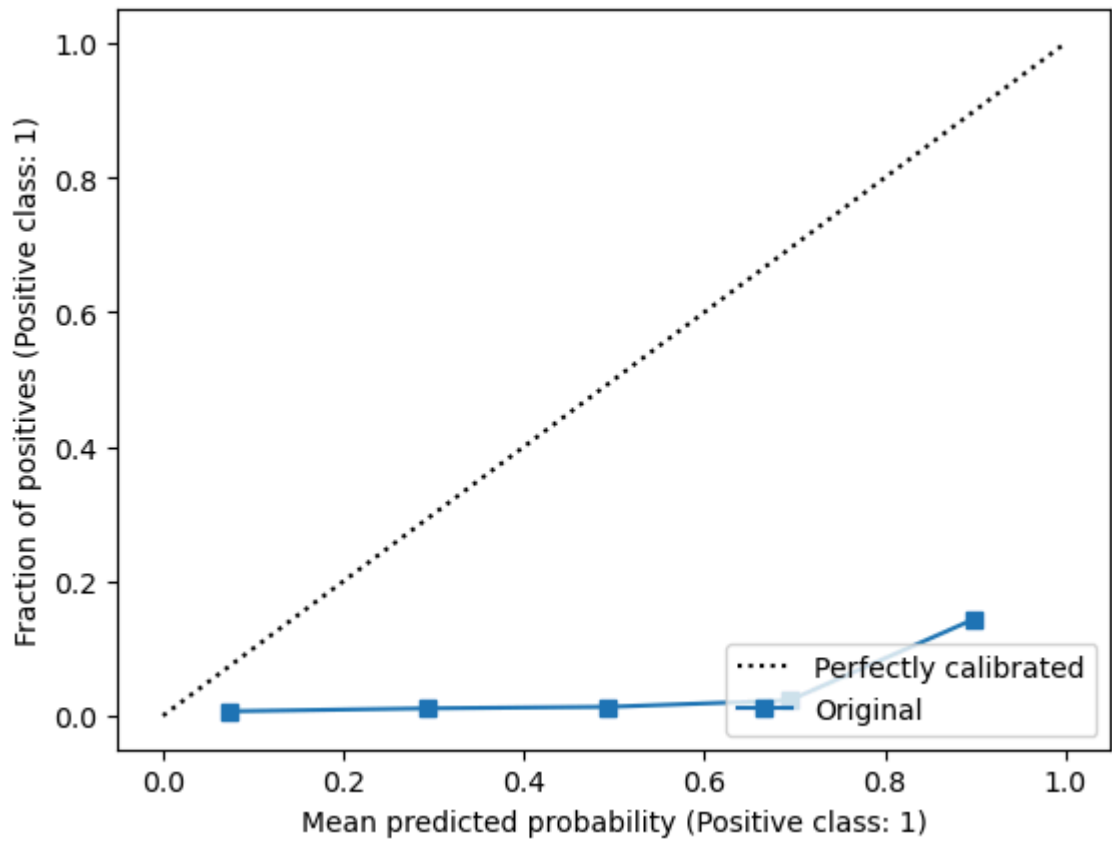
iso_calibrator = CalibratedClassifierCV(lr_model, method='isotonic', cv='prefit')
iso_calibrator.fit(X_train, y_train)

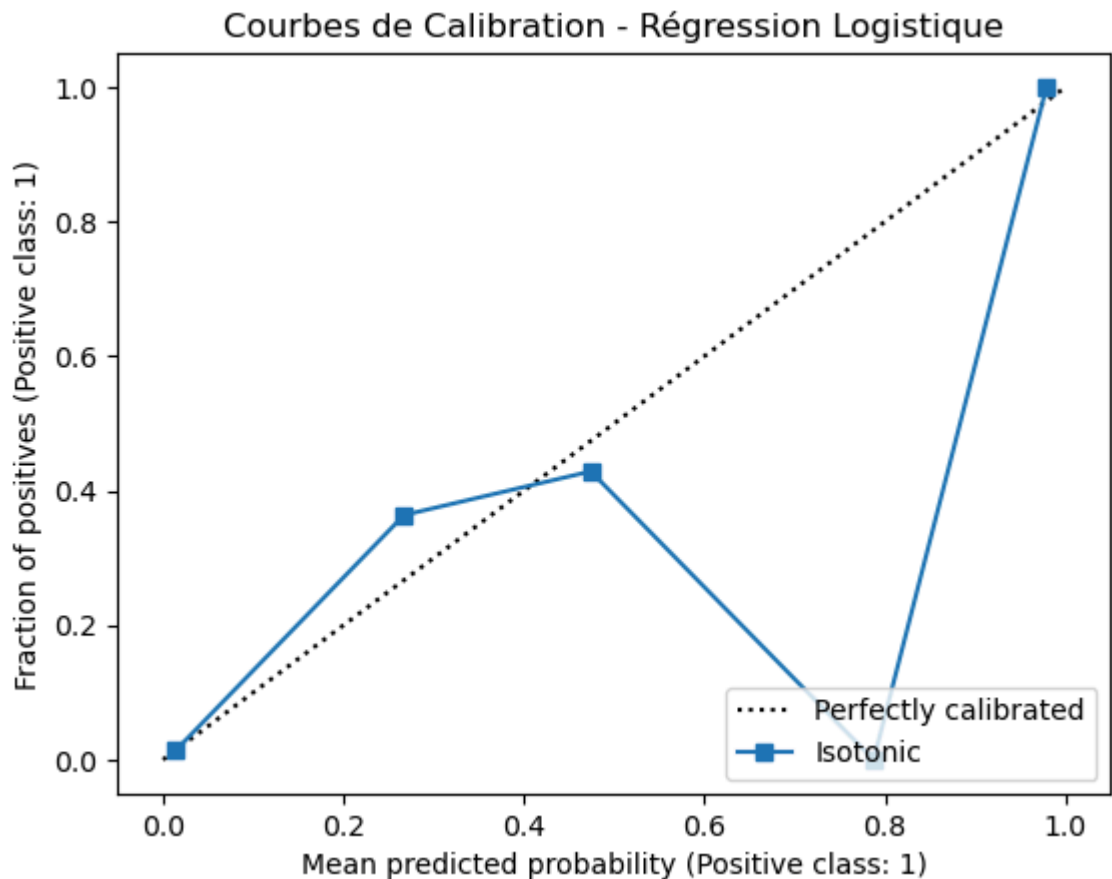
y_proba_iso = iso_calibrator.predict_proba(X_test)[:, 1]

# AFFICHAGE DES COURBES DE CALIBRATION

plt.figure(figsize=(10,6))
CalibrationDisplay.from_predictions(y_test, y_pred_proba_raw, name="Original")
CalibrationDisplay.from_predictions(y_test, y_proba_platt, name="Platt Scaling")
CalibrationDisplay.from_predictions(y_test, y_proba_iso, name="Isotonic")
plt.title("Courbes de Calibration - Régression Logistique")
plt.show()
```

<Figure size 1000x600 with 0 Axes>





```
In [83]: # Sauvegarde
joblib.dump(platt_calibrator, "modele_logistique_calibre_platt.pkl")
```

```
Out[83]: ['modele_logistique_calibre_platt.pkl']
```

```
In [84]: # Chargement
platt_model = joblib.load("modele_logistique_calibre_platt.pkl")
```

```
In [86]: y_pred_calibrated = platt_model.predict(X_test)
y_pred_proba_calibrated = platt_model.predict_proba(X_test)[:, 1]
```

```
In [87]: y_pred_proba_calibrated
```

```
Out[87]: array([0.0272072 , 0.01196996, 0.10321221, ..., 0.00185638, 0.00260713,
0.00257094])
```

Partie 5 : Explicabilité et Fairness (1 semaine)

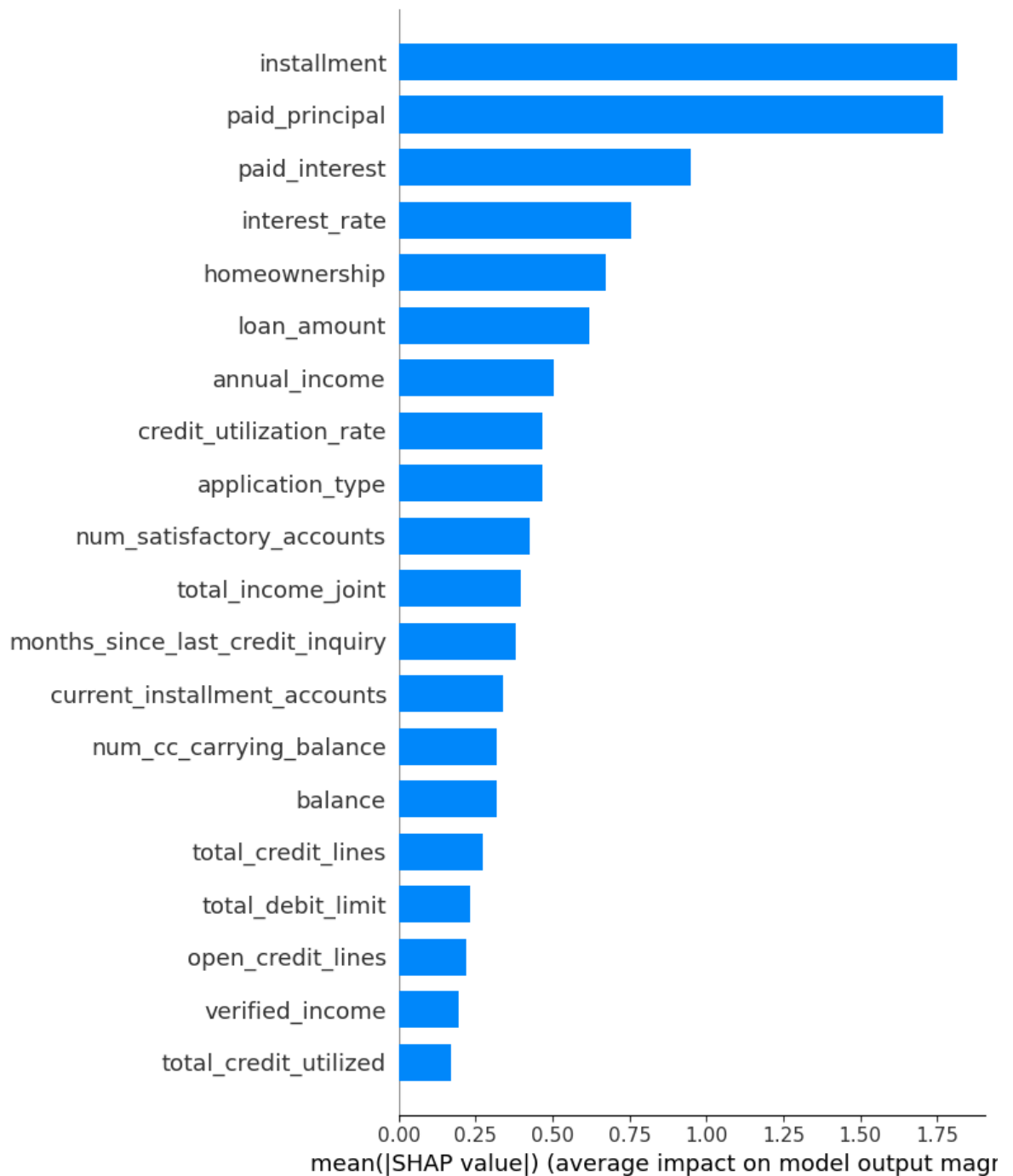
5.1 Explicabilité globale et locale

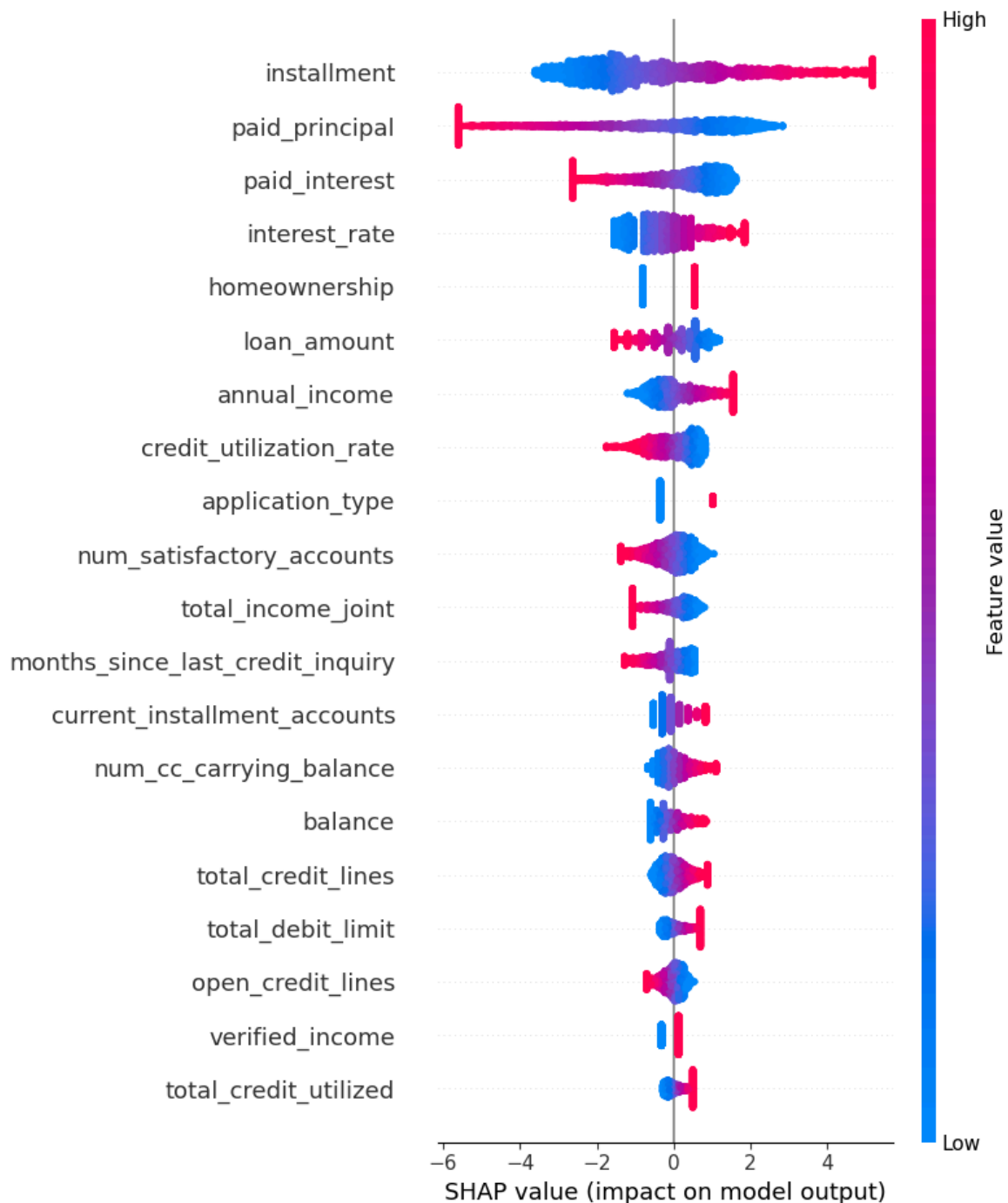
```
In [ ]:
```

```
In [90]: import shap

explainer = shap.LinearExplainer(lr_model, X_train_res)
shap_values = explainer.shap_values(X_test)

shap.summary_plot(shap_values, X_test, plot_type="bar")
shap.summary_plot(shap_values, X_test)
```





In []:

In [91]: `from lime import lime_tabular`

```

# Création de l'explainer LIME
lime_explainer = lime_tabular.LimeTabularExplainer(
    training_data=np.array(X_train_res),
    feature_names=X_train_res.columns,
    class_names=['Bon payeur', 'Mauvais payeur'],
    mode='classification'
)

# Sélection d'un individu du test
i = 5 # ex : 5e individu
exp = lime_explainer.explain_instance(
    data_row=X_test.iloc[i],
    predict_fn=platt_model.predict_proba

```

```
)

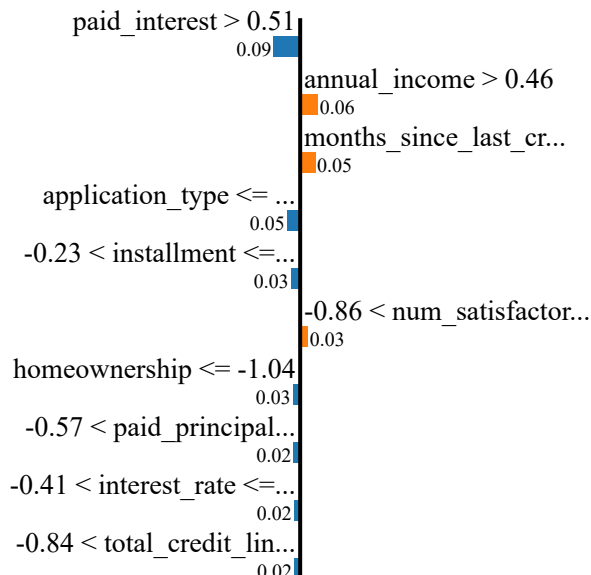
# Affichage de l'explication locale
exp.show_in_notebook(show_table=True)
```

Prediction probabilities

Bon payeur 0.99
 Mauvais payeur 0.01

Bon payeur

Mauvais payeur



Feature Value

paid_interest	0.64
annual_income	0.64
months_since_last_credit_inquiry	-1.15
application_type	-0.42
installment	-0.05
num_satisfactory_accounts	-0.78
homeownership	-1.04
paid_principal	-0.35

```
In [92]: def analyze_credit_fairness(y_true, y_pred, y_proba, X_df):
        """
        Analyse de biais pour les features de crédit
        """
        print("=== ANALYSE DE BIAS POUR MODÈLE DE SCORING CRÉDIT ===\n")

        # Features potentiellement sensibles dans le contexte crédit
        sensitive_features = [
            'homeownership', # Statut de propriété
            'annual_income', # Revenu annuel
            'verified_income', # Revenu vérifié
            'emp_length' # Ancienneté emploi
        ]
```

```

# Filtrage des features disponibles
available_features = [f for f in sensitive_features if f in X_df.columns]
print(f"Features analysées: {available_features}")

results = {}

for feature_name in available_features:
    print(f"\n--- Analyse pour {feature_name} ---")

    feature_values = X_df[feature_name].values

    # Discrétisation basée sur Les quartiles
    if feature_name in ['annual_income', 'emp_length']:
        # Variables continues -> discrétisation
        bins = pd.cut(feature_values, bins=3, labels=['Low', 'Medium', 'High'])
        groups = np.unique(bins)
    else:
        # Variables déjà discrètes
        unique_vals = np.unique(feature_values)
        if len(unique_vals) > 3:
            # Trop de catégories, on regroupe
            median_val = np.median(unique_vals)
            bins = np.where(feature_values <= median_val, 'Low', 'High')
            groups = np.unique(bins)
        else:
            bins = feature_values
            groups = unique_vals

    # Analyse par groupe
    group_analysis = {}
    approval_rates = []

    for group in groups:
        group_mask = (bins == group)
        group_size = np.sum(group_mask)

        if group_size > 0:
            # Taux d'approbation (prédiction = 0 pour bon payeur)
            approval_rate = np.mean(y_pred[group_mask] == 0)
            default_rate_actual = np.mean(y_true[group_mask])
            default_rate_pred = np.mean(y_pred[group_mask] == 1)

            group_analysis[group] = {
                'approval_rate': approval_rate,
                'actual_default_rate': default_rate_actual,
                'predicted_default_rate': default_rate_pred,
                'group_size': group_size,
                'coverage': group_size / len(y_true)
            }
            approval_rates.append(approval_rate)

    print(f" Groupe {group}:")
    print(f" Taux approbation: {approval_rate:.3f}")
    print(f" Taux défaut réel: {default_rate_actual:.3f}")
    print(f" Taille: {group_size} ({group_size/len(y_true):.1%})"

    # Calcul du Disparate Impact
    if len(approval_rates) >= 2:
        min_approval = min(approval_rates)

```

```

        max_approval = max(approval_rates)
        disparate_impact = min_approval / max_approval if max_approval > 0 else 0

        group_analysis['disparate_impact'] = disparate_impact
        group_analysis['fairness_status'] = 'Fair' if disparate_impact >= 0. else 'Unfair'

        print(f" Disparate Impact: {disparate_impact:.3f} ({group_analysis['fairness_status']})")

    results[feature_name] = group_analysis

    return results

def analyze_credit_equalized_odds(y_true, y_pred, X_df):
    """
    Analyse Equalized Odds adaptée au contexte crédit
    """
    sensitive_features = ['homeownership', 'annual_income', 'verified_income']
    available_features = [f for f in sensitive_features if f in X_df.columns]

    fairness_metrics = {}

    for feature_name in available_features:
        feature_values = X_df[feature_name].values

        # Discrétisation
        if feature_name in ['annual_income']:
            bins = pd.cut(feature_values, bins=2, labels=['Low_Income', 'High_Income'])
        else:
            median_val = np.median(feature_values)
            bins = np.where(feature_values <= median_val, 'Low', 'High')

        groups = np.unique(bins)

        if len(groups) < 2:
            continue

        group_metrics = {}

        for group in groups:
            group_mask = (bins == group)
            y_true_group = y_true[group_mask]
            y_pred_group = y_pred[group_mask]

            if len(y_true_group) > 0:
                # Matrice de confusion
                cm = confusion_matrix(y_true_group, y_pred_group)

                if cm.size == 4:
                    tn, fp, fn, tp = cm.ravel()

                    # Taux de faux positifs (bons clients rejetés)
                    fpr = fp / (fp + tn) if (fp + tn) > 0 else 0

                    # Taux de vrais positifs (mauvais clients correctement identifiés)
                    tpr = tp / (tp + fn) if (tp + fn) > 0 else 0

                    # Taux de faux négatifs (mauvais clients acceptés)
                    fnr = fn / (fn + tp) if (fn + tp) > 0 else 0

                    group_metrics[group] = {

```

```

        'fpr': fpr, # Risk of rejecting good clients
        'tpr': tpr, # Ability to detect bad clients
        'fnr': fnr, # Risk of accepting bad clients
        'sample_size': len(y_true_group)
    }

    # Calcul des différences entre groupes
    if len(group_metrics) == 2:
        group_list = list(group_metrics.keys())
        fpr_diff = abs(group_metrics[group_list[0]]['fpr'] - group_metrics[g
        tpr_diff = abs(group_metrics[group_list[0]]['tpr'] - group_metrics[g

        fairness_metrics[feature_name] = {
            'fpr_difference': fpr_diff,
            'tpr_difference': tpr_diff,
            'equalized_odds_violation': max(fpr_diff, tpr_diff),
            'status': 'Fair' if max(fpr_diff, tpr_diff) <= 0.1 else 'Biased'
        }

    return fairness_metrics

def plot_credit_fairness_analysis(fairness_results, equalized_odds_results):
    """
    Visualisation des résultats d'analyse de biais
    """
    fig, axes = plt.subplots(2, 2, figsize=(15, 12))

    # 1. Disparate Impact
    features = list(fairness_results.keys())
    di_values = [fairness_results[f].get('disparate_impact', 1) for f in features]

    colors = ['green' if di >= 0.8 else 'red' for di in di_values]
    axes[0, 0].bar(features, di_values, color=colors, alpha=0.7)
    axes[0, 0].axhline(y=0.8, color='red', linestyle='--', label='Seuil fairness
    axes[0, 0].set_title('Disparate Impact par Feature')
    axes[0, 0].set_ylabel('Disparate Impact Ratio')
    axes[0, 0].tick_params(axis='x', rotation=45)

    # 2. Taux d'approbation par groupe (exemple: homeownership)
    if 'homeownership' in fairness_results:
        home_data = {k: v for k, v in fairness_results['homeownership'].items()
                     if k not in ['disparate_impact', 'fairness_status']}
        groups = list(home_data.keys())
        approval_rates = [home_data[g]['approval_rate'] for g in groups]

        axes[0, 1].bar(groups, approval_rates, color='skyblue', alpha=0.7)
        axes[0, 1].set_title('Taux d\'approbation par Groupe (Homeownership)')
        axes[0, 1].set_ylabel('Taux d\'approbation')

    # 3. Equalized Odds - FPR Difference
    eo_features = list(equalized_odds_results.keys())
    fpr_diffs = [equalized_odds_results[f]['fpr_difference'] for f in eo_features]

    colors_fpr = ['green' if diff <= 0.1 else 'red' for diff in fpr_diffs]
    axes[1, 0].bar(eo_features, fpr_diffs, color=colors_fpr, alpha=0.7)
    axes[1, 0].axhline(y=0.1, color='red', linestyle='--', label='Seuil (0.1)')
    axes[1, 0].set_title('Différence de Faux Positifs (Equalized Odds)')
    axes[1, 0].set_ylabel('Différence FPR')
    axes[1, 0].tick_params(axis='x', rotation=45)

```



```

# 4. Equalized Odds - TPR Difference
tpr_diffs = [equalized_odds_results[f]['tpr_difference'] for f in eo_feature

colors_tpr = ['green' if diff <= 0.1 else 'red' for diff in tpr_diffs]
axes[1, 1].bar(eo_features, tpr_diffs, color=colors_tpr, alpha=0.7)
axes[1, 1].axhline(y=0.1, color='red', linestyle='--', label='Seuil (0.1)')
axes[1, 1].set_title('Différence de Vrais Positifs (Equalized Odds)')
axes[1, 1].set_ylabel('Différence TPR')
axes[1, 1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()

def generate_executive_fairness_report(fairness_results, equalized_odds_results)
    """
    Génère un rapport exécutif sur l'équité du modèle
    """

    print("="*60)
    print("RAPPORT EXÉCUTIF - ANALYSE D'ÉQUITÉ DU MODÈLE DE SCORING")
    print("="*60)

    # Résumé Disparate Impact
    print("\n1. DISPARATE IMPACT ANALYSIS:")
    print("-" * 30)

    biased_features = []
    for feature, results in fairness_results.items():
        di = results.get('disparate_impact', 1)
        status = " CONFORME" if di >= 0.8 else " ATTENTION"
        if di < 0.8:
            biased_features.append(feature)
        print(f"{feature:15} | DI: {di:.3f} | {status}")

    # Résumé Equalized Odds
    print("\n2. EQUALIZED ODDS ANALYSIS:")
    print("-" * 30)

    eo_biased_features = []
    for feature, metrics in equalized_odds_results.items():
        status = " CONFORME" if metrics['equalized_odds_violation'] <= 0.1 else
        if metrics['equalized_odds_violation'] > 0.1:
            eo_biased_features.append(feature)
        print(f"{feature:15} | Violation: {metrics['equalized_odds_violation']:.3f}")

    # Recommendations
    print("\n3. RECOMMENDATIONS:")
    print("-" * 30)

    if not biased_features and not eo_biased_features:
        print(" Le modèle démontre une bonne équité globale.")
        print("Aucune action corrective immédiate nécessaire.")
    else:
        print(" Actions recommandées pour améliorer l'équité:")

        if biased_features:
            print(f" • Features avec biais potentiel: {' '.join(biased_features)}")
            print(" → Envisager un ré-échantillonnage ou re-weighting")

        if eo_biased_features:
            print(f" • Features avec différence de performance: {' '.join(eo_biased_features)}")

```

```
print("      → Envisager l'ajustement des seuils de décision par grou  
print("      • Audit régulier recommandé pour surveiller l'équité")
```

```
In [93]: # EXÉCUTION DE L'ANALYSE  
print("\nDébut de l'analyse d'équité du modèle de scoring...")  
  
# 1. Analyse Disparate Impact  
fairness_results = analyze_credit_fairness(y_test, y_pred, y_proba, X_test)  
  
# 2. Analyse Equalized Odds  
equalized_odds_results = analyze_credit_equalized_odds(y_test, y_pred, X_test)  
  
# 3. Visualisations  
plot_credit_fairness_analysis(fairness_results, equalized_odds_results)  
  
# 4. Rapport exécutif  
generate_executive_fairness_report(fairness_results, equalized_odds_results)  
  
print("\n" + "="*60)  
  
print("="*60)
```

Début de l'analyse d'équité du modèle de scoring...

=== ANALYSE DE BIAS POUR MODÈLE DE SCORING CRÉDIT ===

Features analysées: ['homeownership', 'annual_income', 'verified_income', 'emp_length']

--- Analyse pour homeownership ---

Groupe -1.043129237025198:

Taux approbation: 0.879

Taux défaut réel: 0.021

Taille: 1459 (48.6%)

Groupe 0.9586539850534779:

Taux approbation: 0.718

Taux défaut réel: 0.015

Taille: 1541 (51.4%)

Disparate Impact: 0.817 (Fair)

--- Analyse pour annual_income ---

Groupe High:

Taux approbation: 0.780

Taux défaut réel: 0.027

Taille: 481 (16.0%)

Groupe Low:

Taux approbation: 0.784

Taux défaut réel: 0.010

Taille: 1192 (39.7%)

Groupe Medium:

Taux approbation: 0.814

Taux défaut réel: 0.021

Taille: 1327 (44.2%)

Disparate Impact: 0.958 (Fair)

--- Analyse pour verified_income ---

Groupe -1.3350712102125268:

Taux approbation: 0.891

Taux défaut réel: 0.009

Taille: 1056 (35.2%)

Groupe 0.7490237167505187:

Taux approbation: 0.745

Taux défaut réel: 0.022

Taille: 1944 (64.8%)

Disparate Impact: 0.836 (Fair)

--- Analyse pour emp_length ---

Groupe High:

Taux approbation: 0.817

Taux défaut réel: 0.022

Taille: 1314 (43.8%)

Groupe Low:

Taux approbation: 0.769

Taux défaut réel: 0.013

Taille: 913 (30.4%)

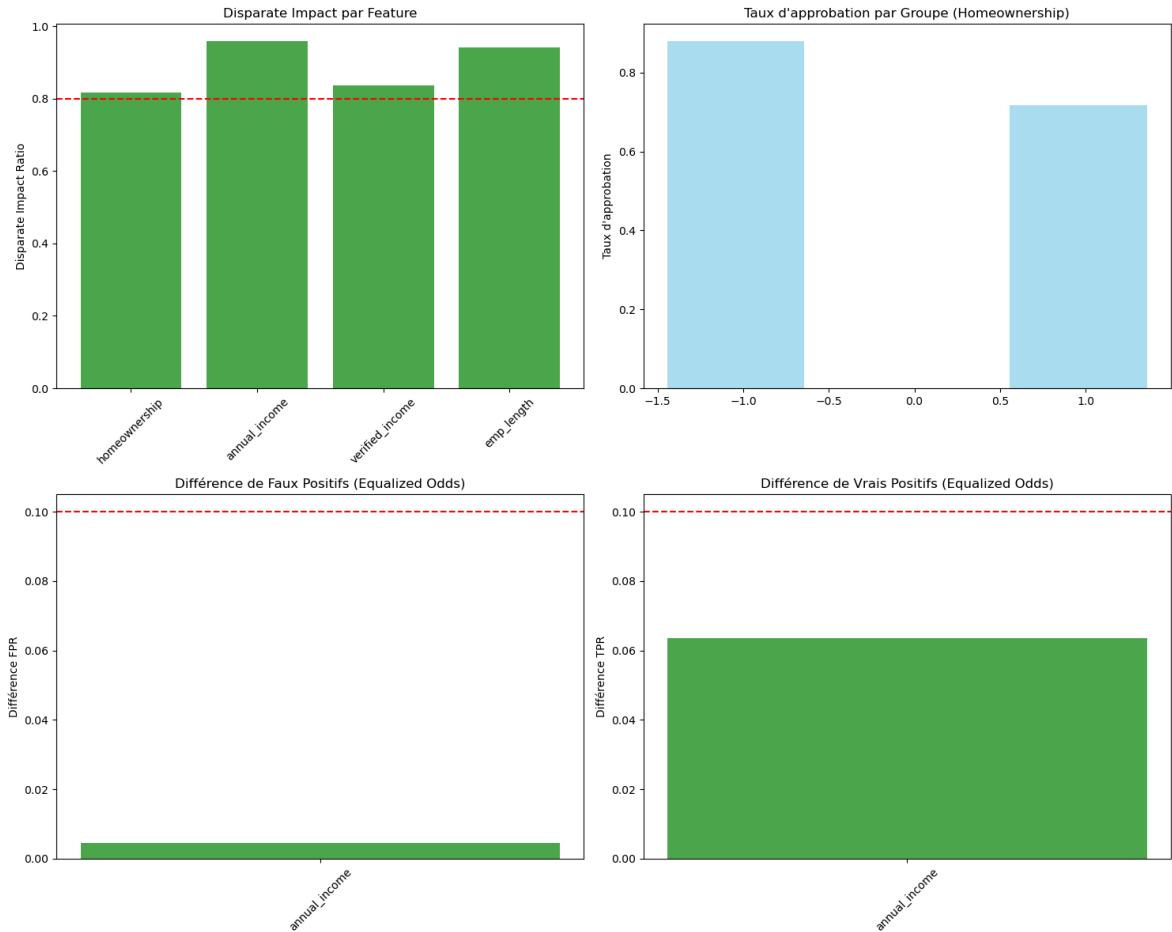
Groupe Medium:

Taux approbation: 0.794

Taux défaut réel: 0.016

Taille: 773 (25.8%)

Disparate Impact: 0.941 (Fair)



=====

RAPPORT EXÉCUTIF - ANALYSE D'ÉQUITÉ DU MODÈLE DE SCORING

=====

1. DISPARATE IMPACT ANALYSIS:

homeownership	DI: 0.817	CONFORME
annual_income	DI: 0.958	CONFORME
verified_income	DI: 0.836	CONFORME
emp_length	DI: 0.941	CONFORME

2. EQUALIZED ODDS ANALYSIS:

annual_income	Violation: 0.064	CONFORME
---------------	------------------	----------

3. RECOMMANDATIONS:

Le modèle démontre une bonne équité globale.
Aucune action corrective immédiate nécessaire.

=====

=====

----- FIN -----
